

PROJET COMPLEXE 2026 REVIVAL

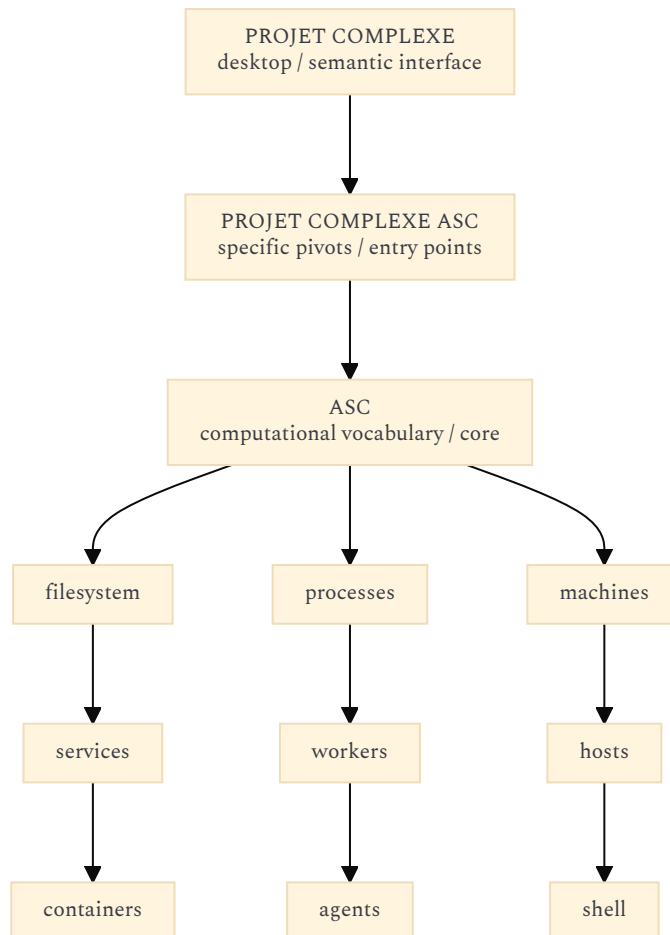
ASC + PROJET COMPLEXE + PROJET COMPLEXE ASC

[ASC](#) and [Projet Complexe](#) started from rather different directions, but they are converging toward a common question:

How can a computational environment become sufficiently explicit, nameable and composable that both humans and autonomous agents can navigate and act within it?

The answer should not be to turn ASC into a second brain, nor to turn Projet Complexe into a system-management application.

The architecture becomes much clearer if three projects are distinguished:



These are not three competing implementations of the same thing.

They are three different scopes.

- **ASC core** defines a generic vocabulary and mechanism for naming, addressing, composing and executing computational things.
- **Projet Complexe** is the desktop application and semantic environment built around tasks, knowledge, research, projects and agents.
- **Projet Complexe ASC** is the deliberately thin integration layer containing the specific ASC entry points, pivots, compositions and environment declarations needed to make Projet Complexe operate through ASC.

The important architectural rule is therefore:

Projet Complexe should use ASC without becoming ASC-specific, while Projet Complexe ASC should use ASC specifically without becoming a second ASC.

This gives each project a reason to exist.

1. THE THREE PROJECTS

1.1 ASC: THE COMPUTATIONAL VOCABULARY

ASC is the most generic layer.

Its concern is not “the second brain”.

Its concern is the computational environment itself:

- files
- directories
- processes
- threads
- machines
- services
- workers

- projects
- commands
- scripts
- environments
- dependencies
- capabilities
- hooks
- entry points
- sidecars
- arguments
- execution
- composition

ASC asks:

What is this thing, where is it, how can it be addressed, and what can be done with it?

It should remain useful even if Projet Complexe never existed.

It should also remain useful if the user never installs a GUI at all.

ASC is therefore not primarily an API server, a desktop backend, a task manager, or a second-brain database.

It is closer to a **language and filesystem-oriented computational model**.

Its ambition is to provide a common vocabulary for anything that interacts with the shell somehow.

That includes things below the shell:

```
machine
OS
filesystem
process
device
service
container
```

and things above it:

```
project
workflow
worker
agent
task
operation
capability
```

without requiring ASC to become specialized for every domain that uses it.

2. PROJET COMPLEXE: THE SEMANTIC AND VISUAL ENVIRONMENT

Projet Complexe has a different concern.

It asks:

What am I trying to accomplish, what do I know, how are things related, and how should I act upon them?

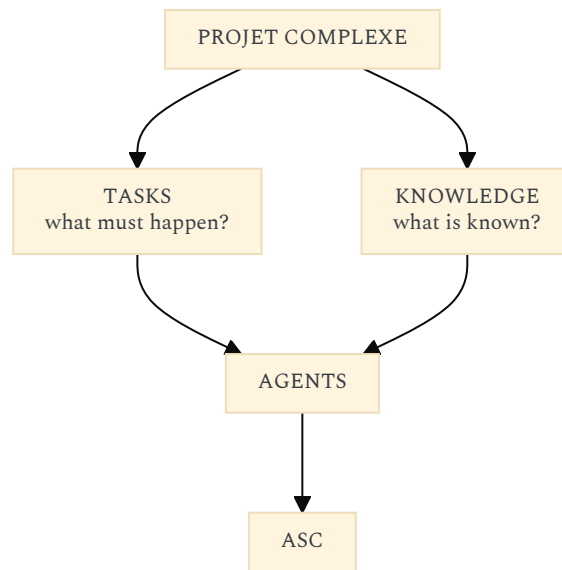
Its vocabulary therefore contains things such as:

```
task
project
objective
plan
idea
document
source
concept
research
knowledge
relationship
publication
agent
```

This is the second-brain layer.

Its desktop application is envisioned as a Tauri + SolidJS application, but the UI technology is not the essential part.

The essential part is the semantic distinction between:



Tasks and knowledge are therefore not merely two navigation tabs.

They are two different orientations of activity.

A task is oriented toward achieving something.

Knowledge is oriented toward understanding something.

Agents sit at the intersection because autonomous activity necessarily moves between the two.

3. PROJET COMPLEXE ASC: THE SPECIFIC PIVOTS

The third project is much thinner.

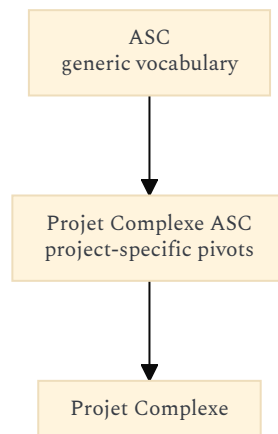
It should not become another application layer.

Its purpose is to contain the **specific ASC-facing vocabulary required by Projet Complexe**.

In other words, Projet Complexe ASC is where we say:

Given the generic capabilities of ASC, these are the particular entry points, compositions, pivots and declarations that this specific environment needs.

Conceptually:



For example, ASC may generically know how to:

```

execute
inspect
search
spawn
stop
watch
read
write
compose
  
```

Projet Complexe ASC can turn those generic mechanisms into specific entry points such as:

```
research
index
extract
recognize
relate
build
run-agent
inspect-agent
stop-agent
publish
```

without requiring those concepts to become primitives of ASC itself.

This is important.

A second brain might need a `research` entry point.

ASC should not therefore become a “research framework”.

The `research` entry point belongs to the layer that composes ASC for that particular domain.

Likewise, a particular environment may need:

```
docling
solr
arangodb
ocr
embedding
llm
agent-runtime
```

Those are not necessarily ASC primitives either.

They are concrete implementations or project-specific pivots.

The distinction can therefore be expressed as:

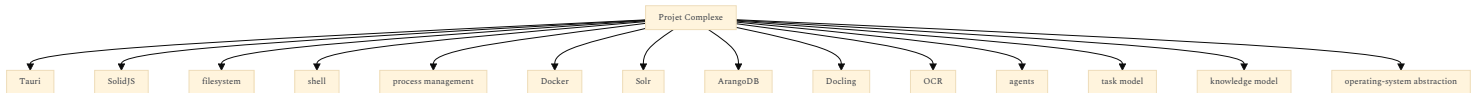
```
ASC
  What is generically possible?

Projet Complexe ASC
  Which of those possibilities does this environment expose,
  and under which stable entry points?

Projet Complexe
  What does all of this mean for the user's tasks,
  knowledge and projects?
```

4. WHY THIS IS BETTER THAN A MONOLITH

The dangerous architecture would be:



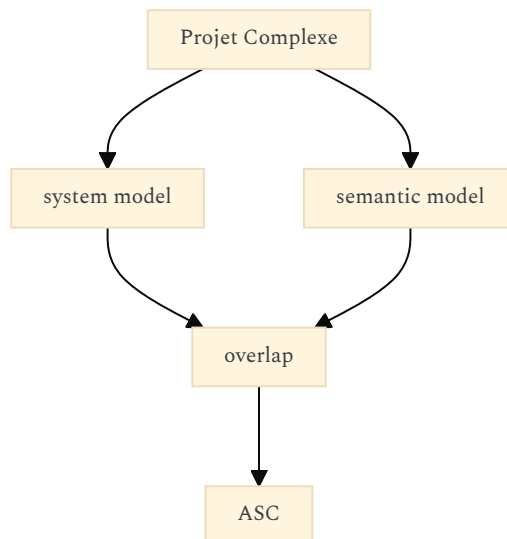
This looks convenient initially.

It eventually creates two problems.

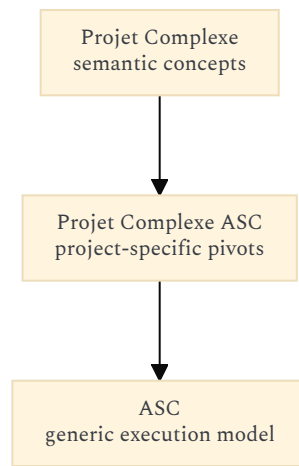
First, the desktop application becomes responsible for things that should belong to ASC.

Second, ASC begins acquiring concepts that only make sense for one application.

The result is duplication:



The three-project architecture prevents this gravitational pull.



Each layer can therefore remain relatively ignorant of the layers above it.

5. ASC IS NOT THE OPERATING SYSTEM

ASC should not attempt to replace Linux, Windows, POSIX, Bash, Docker or other existing computational systems.

Its purpose is to provide a vocabulary *over* them.

For example, “restart this service” may eventually resolve to:

```
systemctl restart foo
```

or:

```
rc-service foo restart
```

or:

```
Restart-Service foo
```

or something completely different.

The abstract operation is stable.

The implementation is not.

This gives ASC several forms of portability:

```
OS portability
Debian
Ubuntu
Arch
Windows
...

tool portability
pdftotext
Docling
Tika
custom parser
...

agent portability
local LLM
remote API
different agent runtime
future backend
```

The consumer should ask for a capability rather than care about the implementation.

6. ASC'S CENTRAL MECHANISM: NAMING AND PIVOTS

The deepest idea in ASC is not “API”.

It is **addressability**.

A thing becomes useful to a computational language when it can be given a stable name and a stable point of access.

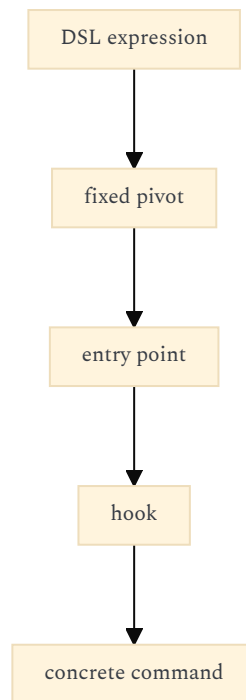
This is why entry points matter so much.

An ASC entry point is a **fixed pivot**.

It is something stable against which an operation can be formulated.

The DSL does not need to recreate arbitrary shell syntax every time.

Instead:



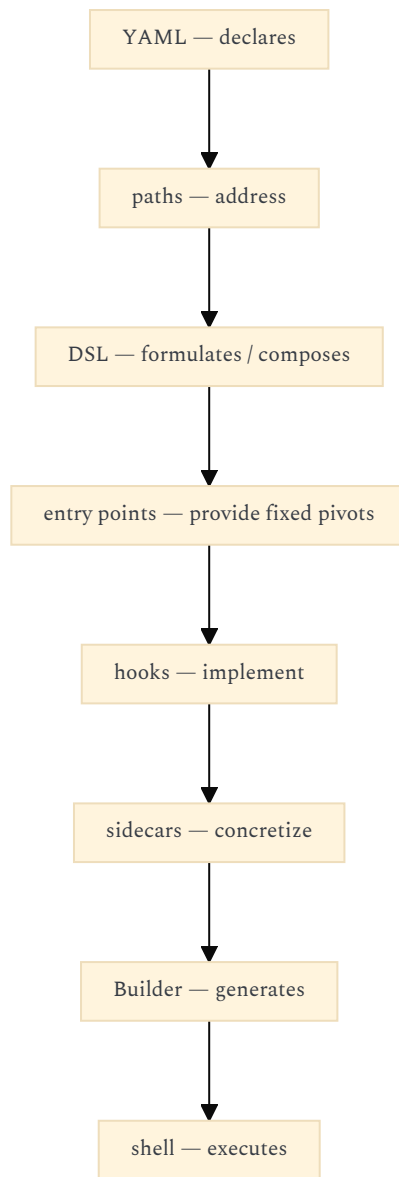
The entry point is therefore not merely a shell script.
It is an addressable semantic pivot.
A concrete implementation can change while the pivot remains stable.
This is one of the places where ASC's filesystem-oriented design becomes interesting.
The same conceptual object can have several representations:

```
YAML declaration
filesystem path
entity
sidecar
entry point
DSL expression
generated artifact
shell execution
graph node
```

These are different views of the same underlying structure.

7. YAML, FILESYSTEM, DSL AND SHELL

ASC can therefore be understood as several complementary representations.



The objective is not to introduce abstraction for abstraction's sake.

The objective is to make the computational structure explicit enough that these representations can correspond to one another.

For example:

```
software.entity.yml
```

can describe a generic kind of software entity.

A concrete entity such as:

```
tesseract.entity.yml
```

can compose that vocabulary.

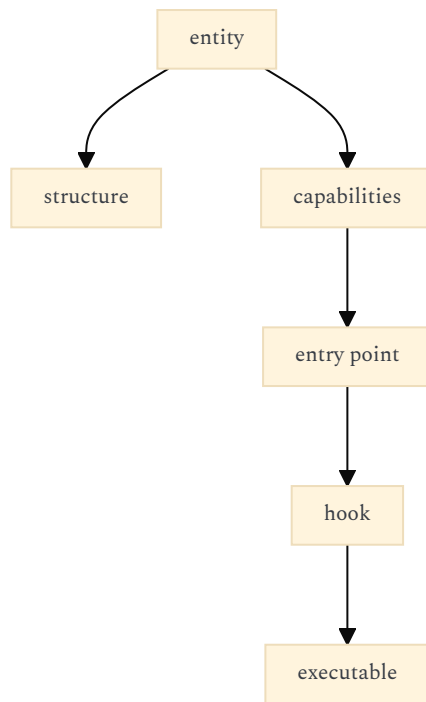
An ability such as:

```
ocr.able.yml
```

can describe a reusable capability.

A concrete hook can then implement the capability.

The conceptual chain becomes:



This is composition rather than conventional object-oriented inheritance.

`include` is therefore better understood as:

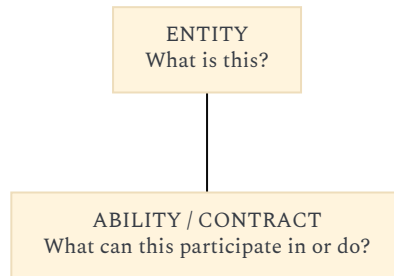
resolve, compose and merge a declaration according to ASC rules.

Not:

class A extends class B.

8. ENTITY VERSUS ABILITY

There is a useful distinction between:



For example:

```

tesseract
  is software

tesseract
  provides OCR

tesseract
  has executable
  
```

The software declaration says what kind of thing it is.

The OCR declaration describes a capability.

That distinction becomes important when agents start exploring environments.

An agent should not need to know:

“This particular program is called Tesseract.”

It should be possible for it to reason more abstractly:

“I need OCR.”

ASC can then discover or resolve an implementation.

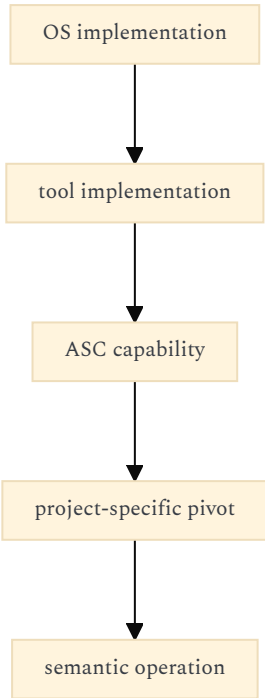
That is a fundamentally different model from hard-coding tool names everywhere.

9. THE PROJECT-SPECIFIC LAYER SHOULD EXPLOIT THIS

Projet Complexe ASC is where these abstract capabilities can be assembled into useful pivots.
For example:

```
recognize-text
index-document
search-knowledge
build-relation
spawn-researcher
run-task
inspect-task
stop-task
publish-document
```

The important point is that these do not have to become universal ASC concepts.
They are **domain compositions built on ASC**.
The project-specific layer can therefore evolve quickly without destabilizing ASC.
If Docling is replaced, the `extract-document` pivot can remain.
If Solr is replaced, `search-knowledge` can remain.
If one agent backend is replaced by another, `spawn-researcher` can remain.
This is the same abstraction principle operating at several scales:



10. PROJECT COMPLEXE SHOULD REMAIN IGNORANT OF IMPLEMENTATION DETAILS

The UI should not think in terms of:

```
SolrService
ArangoService
DockerService
LinuxService
PythonService
TesseractService
```

It should think in terms of:

```
Search
Graph
Machine
Project
Worker
Agent
Task
Document
Source
Knowledge
```

This distinction matters because the semantic vocabulary should remain stable even when the technology changes.
The user should not have to know that:

```
Search
```

currently happens through Solr.
Nor that:

Document extraction

currently happens through Docling.
Nor that:

Graph

currently happens through ArangoDB.
Those are implementation facts.
The semantic model should outlive them.

11. PROJET COMPLEXE IS THEREFORE NOT A SYSTEM DASHBOARD

This distinction is crucial.
The application should not become:

CPU: 47%
RAM: 62%
DISK: 81%
Docker: OK
Solr: OK
ArangoDB: OK

simply because ASC makes those things observable.
That information can be useful, but it is not the application's purpose.
The interface should instead become a **visual query surface over a complex system**.
For example:



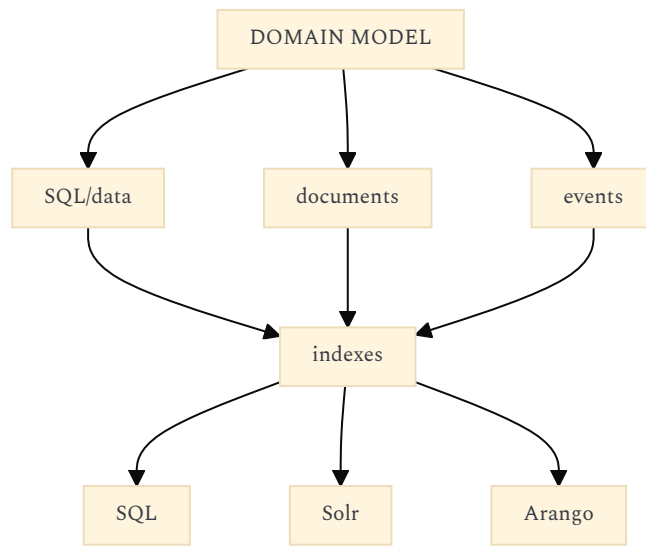
A machine can appear because it is relevant to a project.
A process can appear because it belongs to an agent run.
A document can appear because it is evidence for a task.
A service can appear because it constrains an operation.
Everything is contextual.

12. THE GRAPH IS CONCEPTUAL, NOT NECESSARILY THE DATABASE

The system may contain:

relational data
documents
files
events
logs
search indexes
graph data

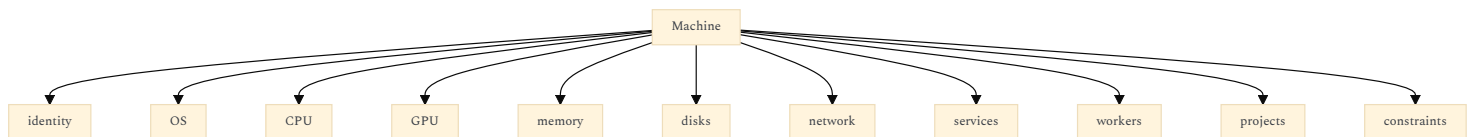
There is no requirement that everything physically live in one graph database.
The conceptual graph can instead be projected from several stores:



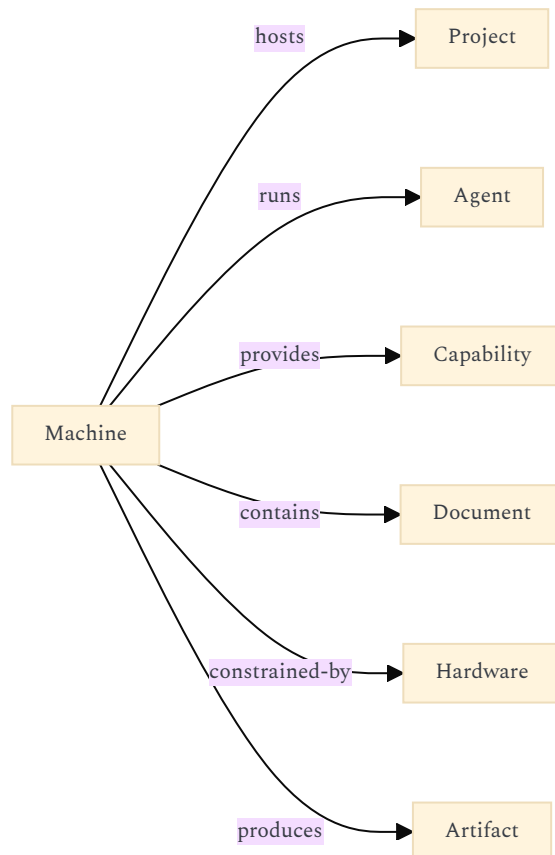
The graph is therefore a model of relationships.
 It is not necessarily the storage architecture.
 This distinction preserves implementation freedom.

13. MACHINES BECOME KNOWLEDGE OBJECTS

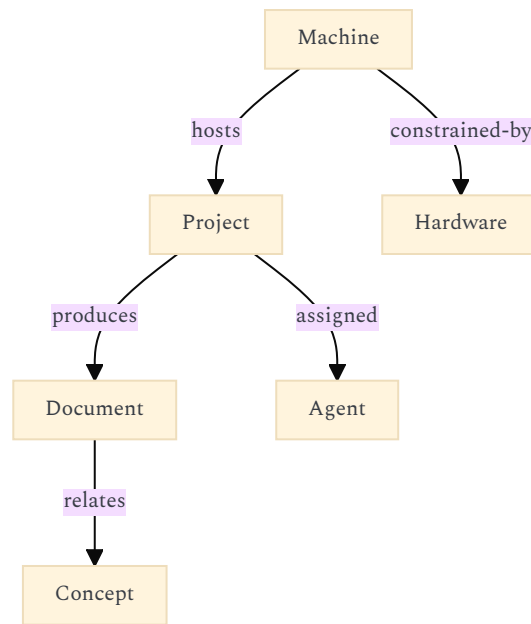
ASC can expose the computational world.
 Projet Complexe can interpret it.
 A machine therefore becomes more than a hardware dashboard.
 Conceptually:



Projet Complexe can then relate that machine to semantic objects:



The result is a unified model:



This is where ASC and the second brain become complementary without becoming the same project.
 ASC tells us what is observable and executable.
 Project Complexe gives it semantic meaning.

14. THE CENTRAL DISTINCTION: EXECUTION VERSUS INTERPRETATION

A useful formulation is:

ASC is authoritative about execution. Project Complexe is authoritative about interpretation.

ASC answers:

What exists?
 What can be done?
 Where is it?
 How is it executed?

Project Complexe answers:

Why does it matter?
 What is it related to?
 What am I trying to accomplish?
 What do I currently believe?
 What should happen next?

The boundary is therefore not:

backend vs frontend

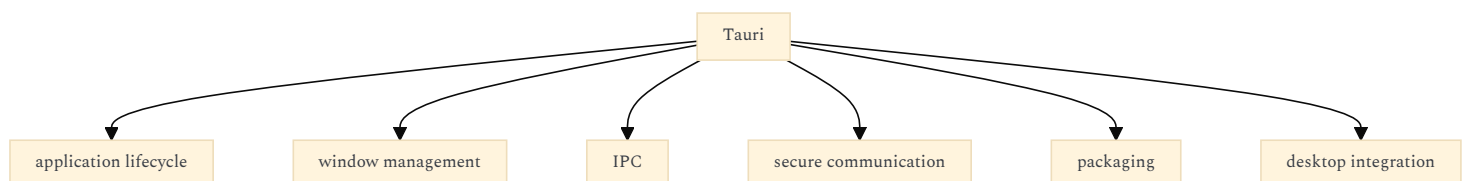
It is:

execution vs interpretation

The Tauri application is simply one interface through which the interpretive layer can interact with the execution layer.

15. TAURI SHOULD REMAIN THIN

The Tauri application should not recreate ASC.
 Its responsibilities are approximately:



SolidJS provides:

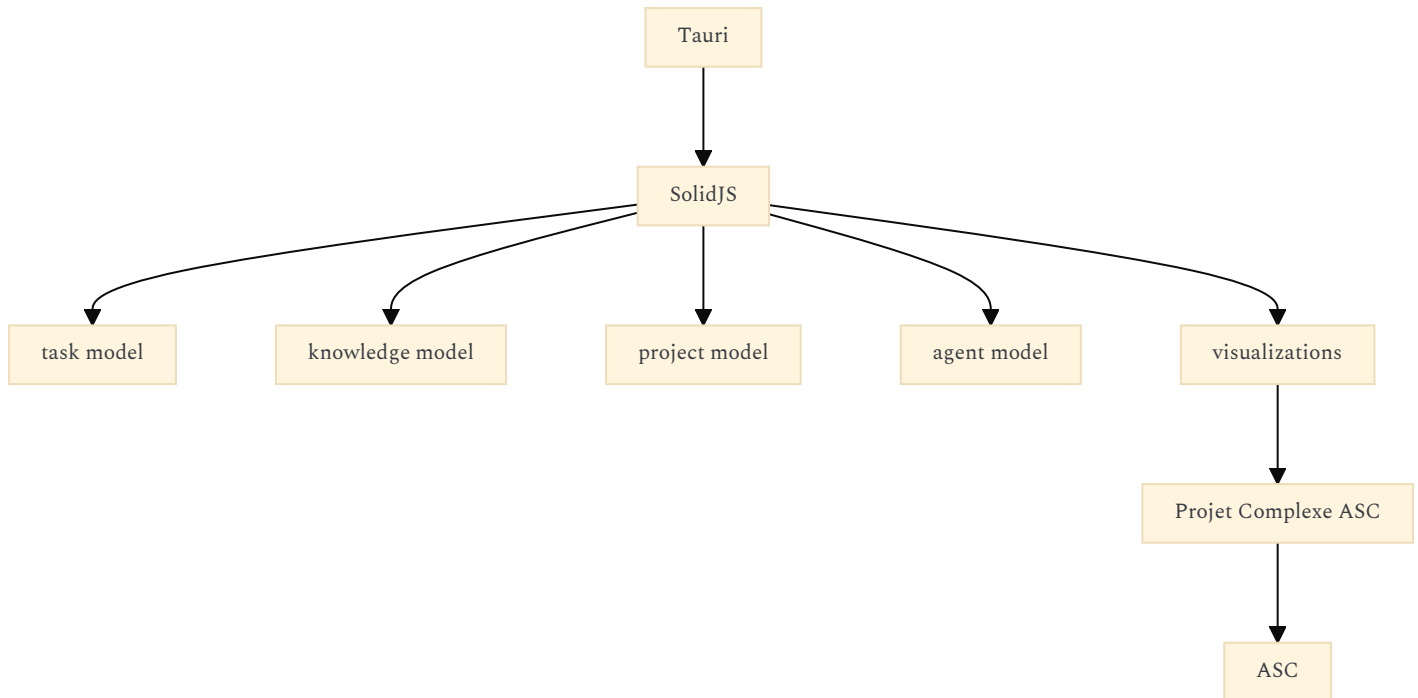
views
editors
graph visualization
task interfaces
knowledge interfaces
agent visualization
machine visualization

Kobalte provides behavioral and accessibility primitives.

CSS provides the visual language.

ASC provides the computational substrate.

The application therefore becomes:



This keeps the Rust portion deliberately small.

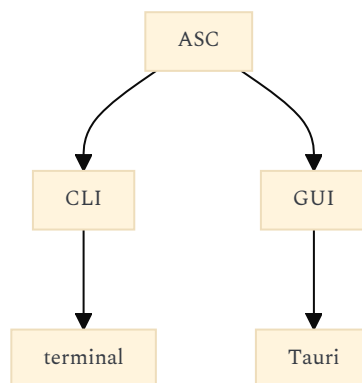
16. CLI AND GUI SHOULD BE TWO INTERFACES TO THE SAME SYSTEM

One of the strongest consequences of this architecture is that the GUI should never become the only way to operate the system.

The invariant should be:

Anything the GUI can cause to happen should remain reproducible from the terminal.

Conceptually:



If the UI says:

Indexing failed

the underlying operation should still be inspectable and reproducible through ASC.

This is important for debugging, automation, scripting and agents.

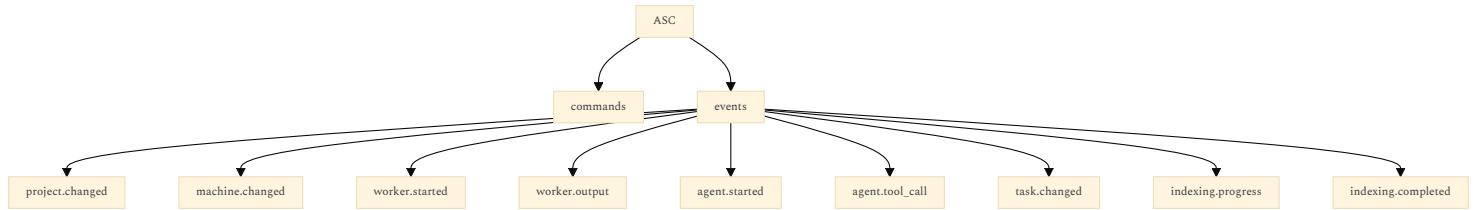
It also prevents the desktop application from becoming a mysterious execution environment that cannot be understood outside itself.

17. EVENTS SHOULD BE FIRST-CLASS

The GUI should not need to poll constantly:

Tauri → ASC → status
Tauri → ASC → status
Tauri → ASC → status

The natural model is:



The desktop application subscribes.

The CLI can subscribe.

An external client can subscribe.

An agent controller can subscribe.

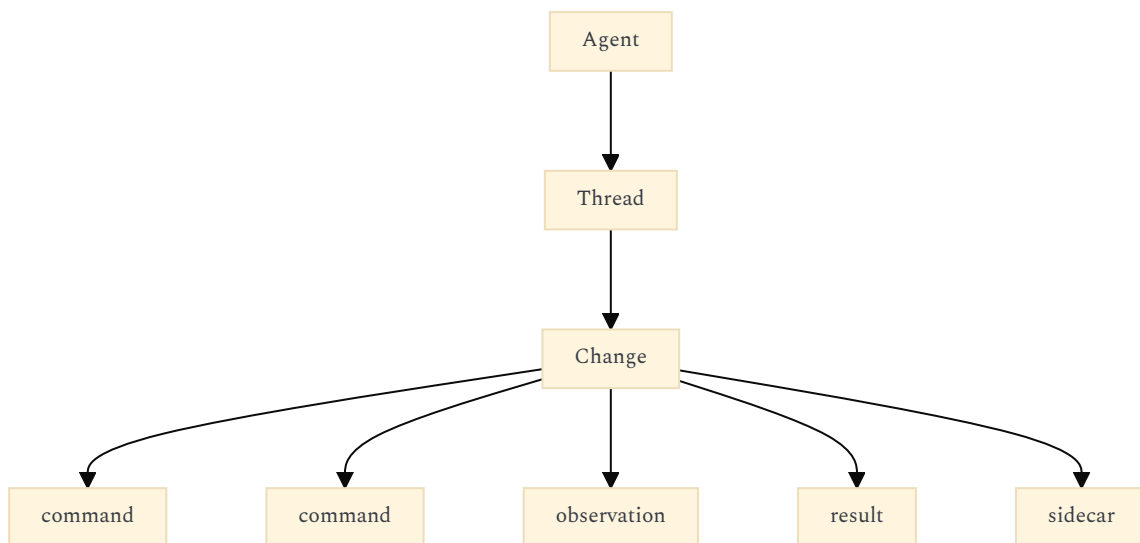
This means that execution becomes observable without forcing every consumer to understand the implementation.

18. AGENTS ARE NOT ANOTHER SILO

Agents should not become another independent object model disconnected from tasks and knowledge.

An agent is a computational actor operating inside this environment.

A useful conceptual model is:



The important thing is that agent activity becomes visible as activity in the same environment.

An agent does not have to be treated as magical.

It can be represented through:

intent
plan
tool call
observation
command
result
change
state transition
output

This is precisely where ASC's computational vocabulary becomes useful.

19. THE TASK/KNOWLEDGE DISTINCTION IS DEEPER THAN A UI DISTINCTION

The original Projet Complexe distinction between **task-oriented** and **knowledge-oriented** work should remain central.

But it should not be interpreted as:

Task tab
Knowledge tab

It describes two different modes of autonomous activity.

A task has an imperative structure:

objective
state
dependencies
inputs
outputs
deadline
agent
execution

Knowledge has an epistemic structure:

claim
source
evidence
concept
relationship
confidence
provenance
context

They intersect continuously.

A task may require knowledge.

A task may produce knowledge.

Knowledge may invalidate a task.

A task may reveal that knowledge is missing.

This is where the more interesting idea begins.

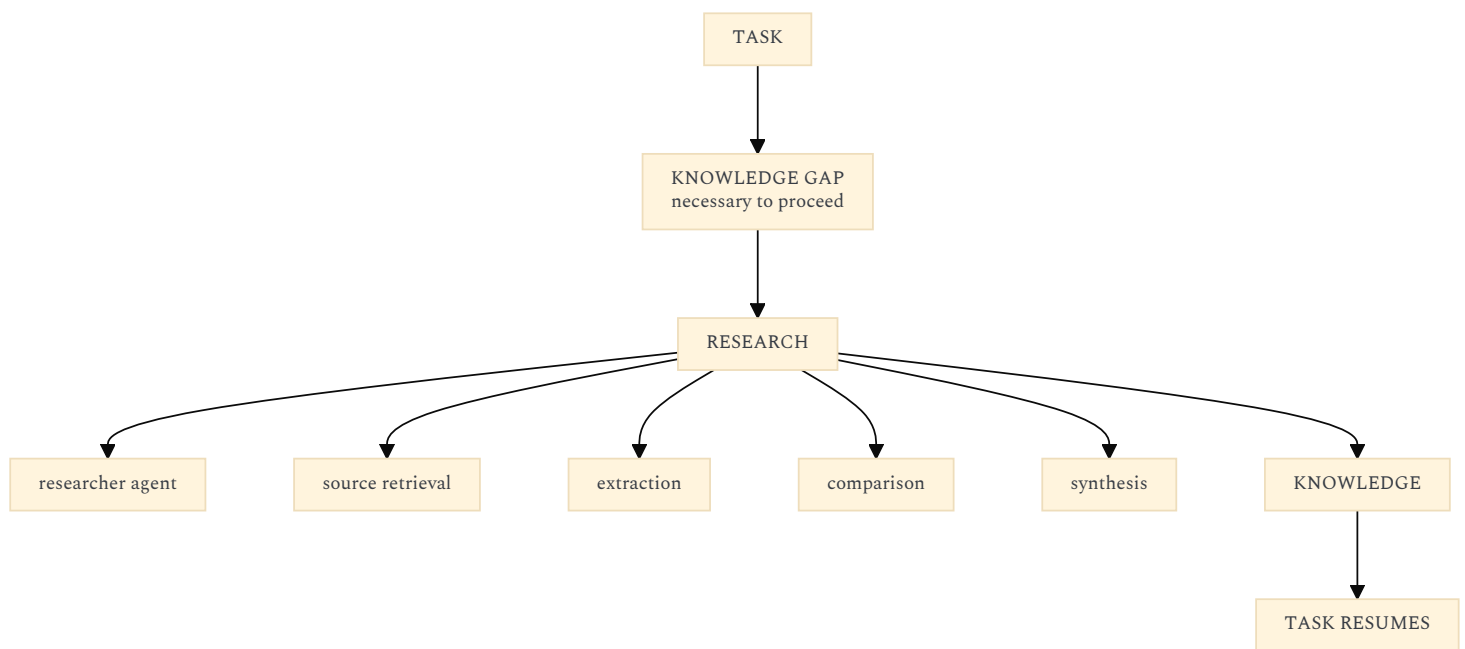
20. THE TASK/KNOWLEDGE MUTUAL KILLSWITCH

The phrase “**task-oriented vs knowledge-oriented: mutual killswitch**” should be understood literally as a control principle for autonomous agents.

It does **not** mean that tasks and knowledge are simply synchronized.

It means that each can impose a stopping condition on the other.

The first direction is:



A task can therefore **kill or suspend itself** because it discovers that it lacks knowledge necessary to continue responsibly.

The task says, in effect:

I cannot legitimately proceed from what I currently know.

This is not failure.

It is a controlled transition from execution to inquiry.

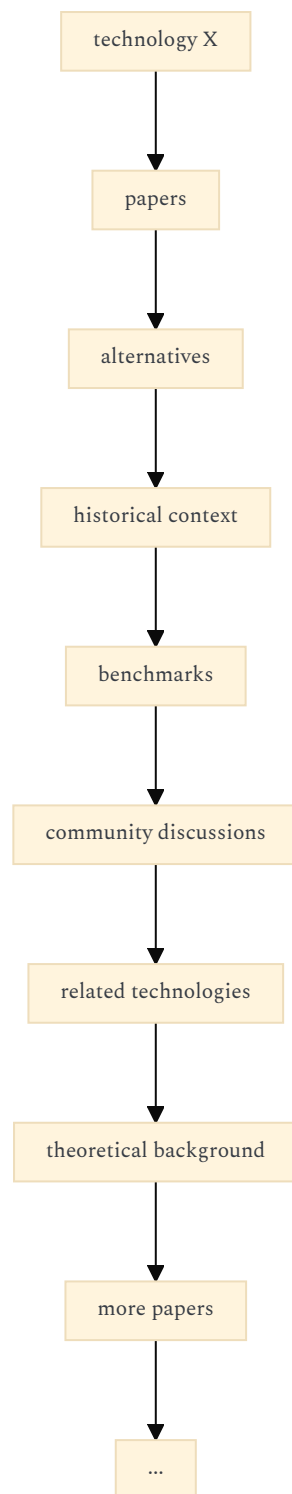
21. THE OPPOSITE DIRECTION IS EQUALLY IMPORTANT

Knowledge gathering can itself become pathological.

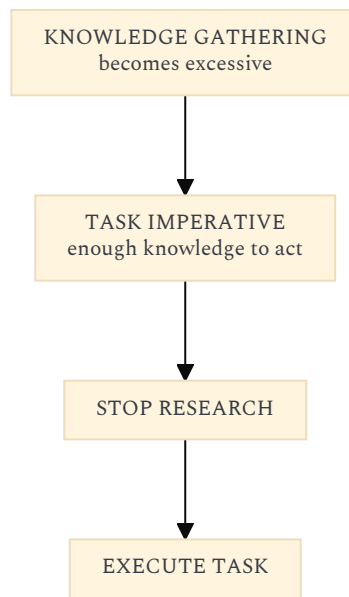
Suppose an agent is asked to accomplish:

Determine whether technology X is appropriate for this project.

It may begin researching:



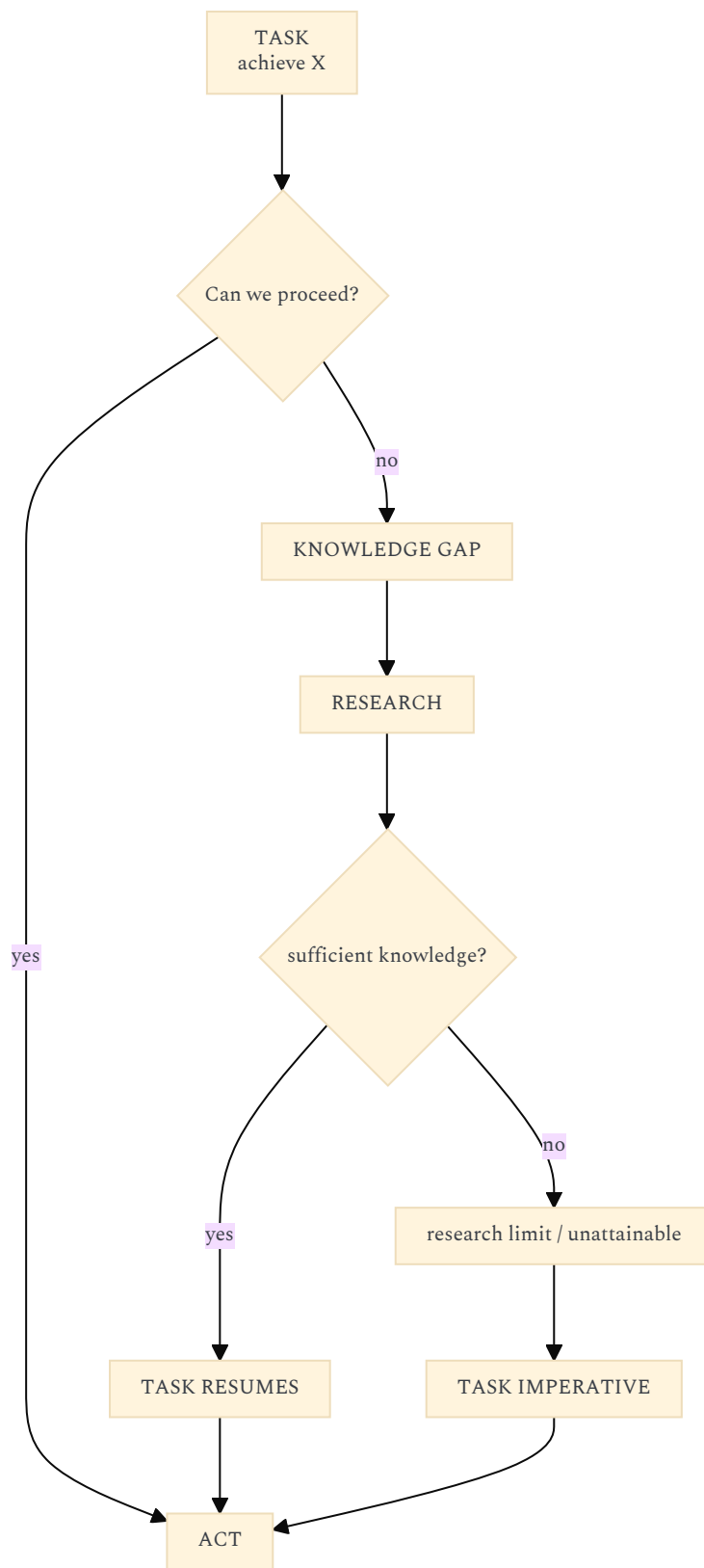
At some point, the research process can become effectively unbounded.
The system then needs the opposite control:



Knowledge gathering must therefore be killable by the task it exists to serve.
This is the second direction of the mutual killswitch.

22. THE COMPLETE CONTROL LOOP

The resulting loop is:



The critical concept is **sufficiency**.

The goal of research is not:

know everything.

It is:

know enough to make the next justified move.

Likewise, the goal of task execution is not:

continue regardless of uncertainty.

It is:

make progress until a knowledge deficit becomes materially blocking.

23. THIS CHANGES WHAT “AUTONOMOUS” MEANS

An autonomous agent should not be defined merely as:

an agent that keeps executing without human intervention.

That model encourages runaway behavior.

A more useful definition is:

an agent that can regulate the relationship between action and uncertainty.

It needs to know when:

I can act.

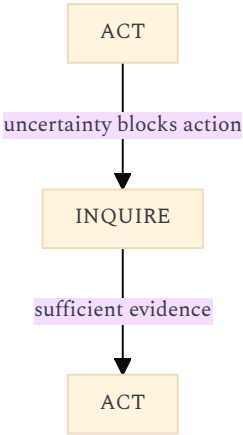
when:

I cannot act without learning something.

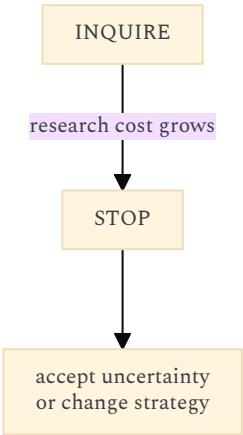
and when:

I have learned enough; further learning is no longer justified by the task.

That gives us a three-state epistemic/action loop:



with a second constraint:



The task therefore supplies a bounded purpose to knowledge.

Knowledge supplies a bounded legitimacy to action.

24. THIS IS PARTICULARLY RELEVANT TO RESEARCHER AGENTS

A researcher agent should not simply have the objective:

“Find everything about X.”

It should have a task context.

For example:

Parent task:
Choose an extraction strategy for this corpus.

Knowledge requirement:
Determine whether Docling handles the relevant PDF structures.

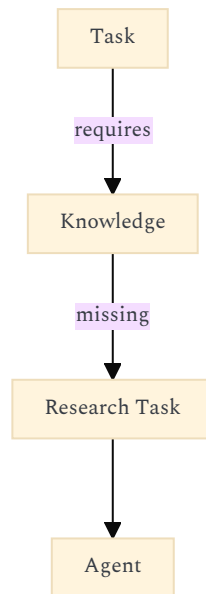
Research task:
Investigate Docling's capabilities.

Stopping conditions:
sufficient evidence for decision
OR research budget exhausted
OR question shown to be undecidable

The researcher is therefore subordinate to the parent task.
It exists because something needs to be accomplished.
This prevents knowledge acquisition from becoming an independent optimization target.

25. THE SAME PRINCIPLE APPLIES TO AGENTS RESEARCHING AGENTS

The system should avoid a hierarchy where every unknown automatically spawns another infinite research tree.
A knowledge gap should be represented explicitly:



The research task has its own budget and stopping conditions.
If it cannot produce useful knowledge within those constraints, it should report:

knowledge unavailable

rather than silently expanding forever.
This creates a natural form of bounded autonomy.

26. KNOWLEDGE IS THEREFORE NOT MERELY A DATABASE

This is one reason Projet Complexe should not be reduced to:

notes
documents
embeddings
vector database
graph database
RAG

Those are storage and retrieval mechanisms.
The actual knowledge model concerns:

claims
sources
evidence
relationships
uncertainty
provenance
context
questions
unknowns

The most interesting knowledge object may sometimes be:

Unknown

or:

KnowledgeGap

because that object can directly influence task execution.

27. THE KNOWLEDGE GRAPH SHOULD CONTAIN UNCERTAINTY

A semantic environment becomes more useful to autonomous agents when it can represent not only:

X is true.

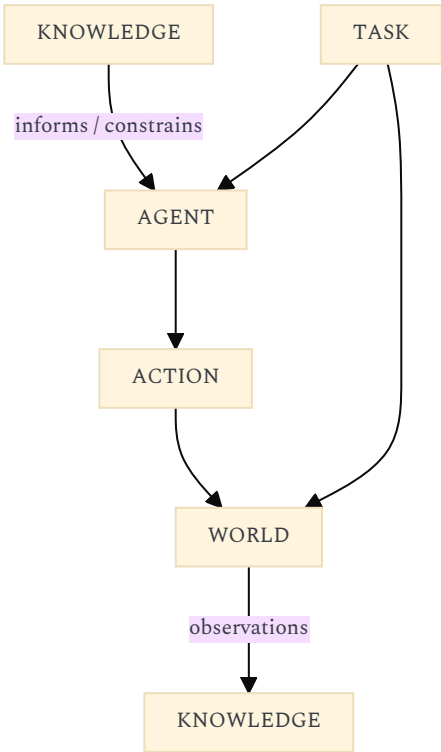
but also:

X is suspected.
X is supported by source Y.
X conflicts with Z.
X remains unknown.
X is irrelevant to task T.
X is sufficient for decision D.

This makes the knowledge layer operational.
It stops being merely an archive and becomes part of the agent's control system.

28. TASKS AND KNOWLEDGE FORM A COUPLED SYSTEM

The deeper model is therefore:



Knowledge constrains action.
Action generates observations.
Observations modify knowledge.
Knowledge gaps interrupt action.
Excessive knowledge gathering is interrupted by the task imperative.
This is a feedback system rather than a pipeline.

29. THE “MUTUAL KILLSWITCH” IS THEREFORE A CONTROL-THEORETIC IDEA

The term is intentionally stronger than “feedback loop”.
A feedback loop merely describes interaction.
A killswitch introduces authority.

The task has authority to say:

stop researching; act.

The knowledge requirement has authority to say:

stop acting; research.

This gives the two modes competing but complementary imperatives:

TASK:
achieve something.

KNOWLEDGE:
know enough to act responsibly.

Neither should permanently dominate.
A purely task-oriented system risks blind execution.
A purely knowledge-oriented system risks endless inquiry.
The mutual killswitch is the mechanism that keeps them coupled without allowing either to become sovereign.

30. THIS MAY BECOME ONE OF PROJET COMPLEXE'S MOST IMPORTANT CONCEPTS

The original visual division between:

TASK

and:

KNOWLEDGE

can therefore evolve into something much more fundamental.
The interface could make the current mode explicit:

TASK

Objective:
decide extraction strategy

State:
blocked

Reason:
insufficient evidence

Knowledge gap:
PDF structure compatibility

Research:
active

Then, later:

KNOWLEDGE

Question:
Can Docling handle this corpus?

Evidence:
7 sources

Confidence:
sufficient for current decision

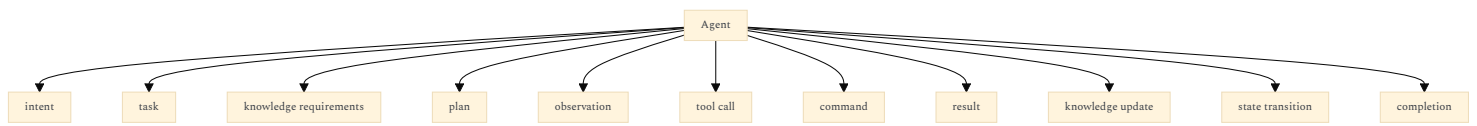
Research:
terminated by task requirement

TASK:
resumed

The UI is not merely displaying work.
It is displaying the system's **epistemic state**.

31. AGENT ACTIVITY SHOULD BECOME AN EVENT STREAM

An agent run can therefore be represented as:



For example:

```

Agent: research-agent

10:42 TASK
Determine whether PDF extraction is feasible.

10:42 CHECK KNOWLEDGE
Required capability: structured PDF extraction.

10:42 KNOWLEDGE GAP
Current corpus contains unsupported structures.

10:43 SPAWN
researcher-agent

10:43 SEARCH
local corpus

10:44 EXTRACT
document A

10:45 COMPARE
document B

10:47 KNOWLEDGE UPDATE
evidence sufficient for decision

10:47 KILL RESEARCH
parent task resumes

10:48 TASK
select extraction strategy
  
```

This is much more informative than simply showing:

```

agent running
  
```

32. ASC SHOULD EXPOSE EXECUTION EVENTS, NOT INTERPRET THEM

ASC can provide:

```

agent.started
agent.tool_call
process.started
process.output
file.changed
worker.completed
thread.stopped
  
```

Projet Complexe can interpret those events semantically:

```

research started
knowledge gap discovered
evidence added
task blocked
task resumed
decision reached
  
```

This is another important boundary.
 ASC reports what happened.
 Projet Complexe determines what it means.

33. THREADS, PROCESSES AND AGENTS

ASC should not necessarily mirror operating-system terminology exactly.
 An ASC `thread` can represent an ASC-managed execution pivot.
 It does not need to mean literally:

```

| POSIX thread.
  
```

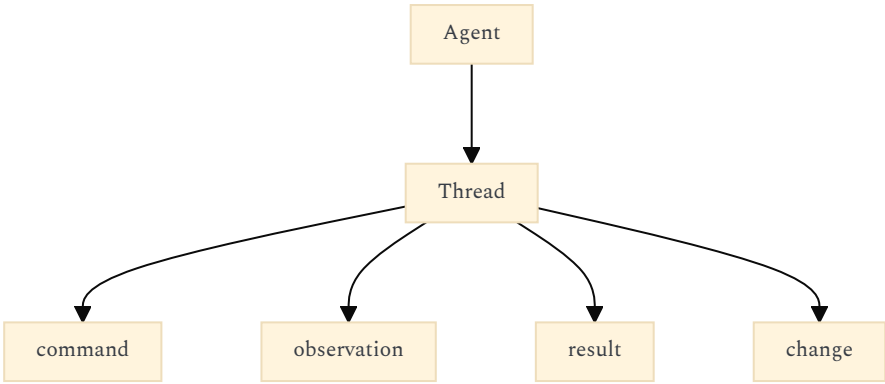
Likewise, an agent is not simply:

```

| a process.
  
```

An agent may cause many processes to exist.

It may spawn workers.
It may create files.
It may launch commands.
It may invoke remote services.
The semantic object is therefore:



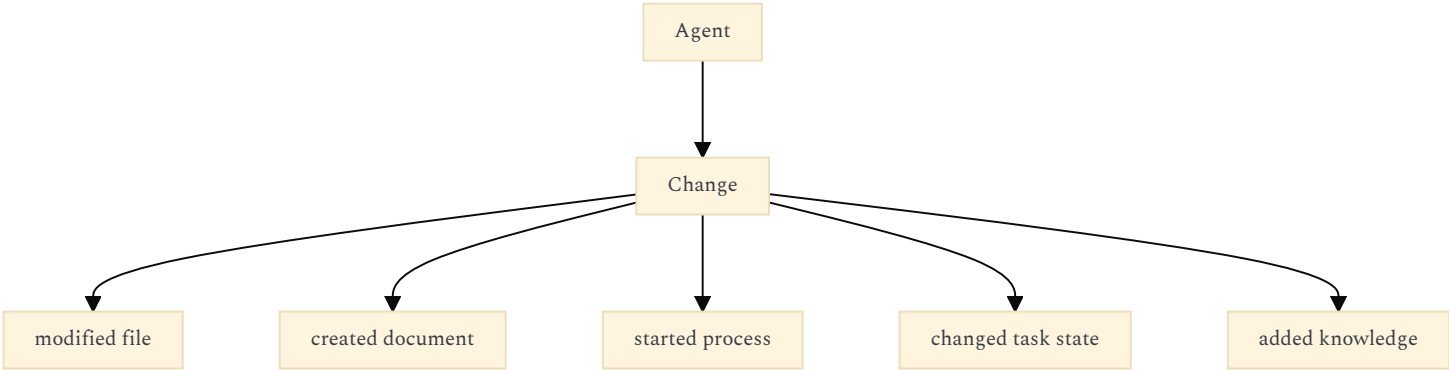
while the implementation may involve:

```
processes
containers
remote calls
shell commands
APIs
```

ASC provides the pivot through which those implementation details become addressable.

34. CHANGES ARE PARTICULARLY IMPORTANT

A `change` can become a bridge between execution and knowledge.
For example:



This makes autonomous work auditable.
Instead of thinking:

the agent did some things,

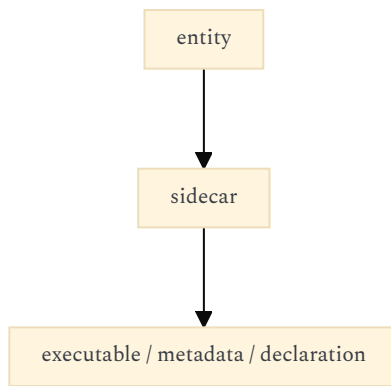
the system can represent:

this agent produced these observable changes.

That is an important foundation for trustworthy autonomous systems.

35. SIDECARS MAKE VIRTUAL THINGS CONCRETE

A recurring ASC pattern is that a conceptual object can have a concrete representation.
A sidecar can provide that representation.
This is particularly useful when the thing being represented is not itself naturally a file.
For example:



The same pattern can apply to agents, threads, capabilities, software and other computational objects.
The sidecar is not necessarily the object itself.
It is a concrete projection of the object.

36. THE FILENAME-SAFE CONSTRAINT MATTERS

If the filesystem is part of the vocabulary, names need to remain safe and predictable.
The same is true for DSL-generated entry points.
A fixed pivot should not depend on arbitrary characters that are difficult to represent consistently across:

```
filesystem
shell
environment variables
DSL
URLs
logs
JSON
```

This constraint is not cosmetic.
It is part of making the vocabulary composable.

37. THE DSL SHOULD REMAIN SMALL

The DSL should not become a replacement for Bash.
Its purpose is to provide a compact notation for addressing and composing ASC's fixed pivots.
The important distinction is:

```
Bash:
  general-purpose shell language

ASC DSL:
  structured notation for ASC vocabulary and pivots
```

The DSL therefore benefits from being deliberately constrained.
The more it becomes another general-purpose programming language, the less useful the relationship with the filesystem and entry-point model becomes.

38. ARGUMENT MAPPING BELONGS TO THE PIVOT MODEL

ASC can expose positional and option slots through stable names.
The goal is not necessarily to reproduce Bash's:

```
$1
$2
$@
```

directly.
The goal is to give those positions semantic names.
This allows a DSL expression to remain stable while the implementation underneath it changes.
The entry point establishes interpretation.
The DSL supplies the composition.
The shell receives the final concrete arguments.

39. wrap, nest AND COMPOSITION

The distinction between wrapping and nesting is useful because composition happens at different levels.
Conceptually:

invoke
execute an existing pivot

wrap
expose existing behavior through another interface

nest
compose one computational structure inside another

The important property is that composition should not require copying implementation.
This is the same principle that makes the entity system interesting.
ASC should favor:

compose

over:

duplicate

40. PARALLELISM SHOULD BE A PROPERTY OF COMPOSITION

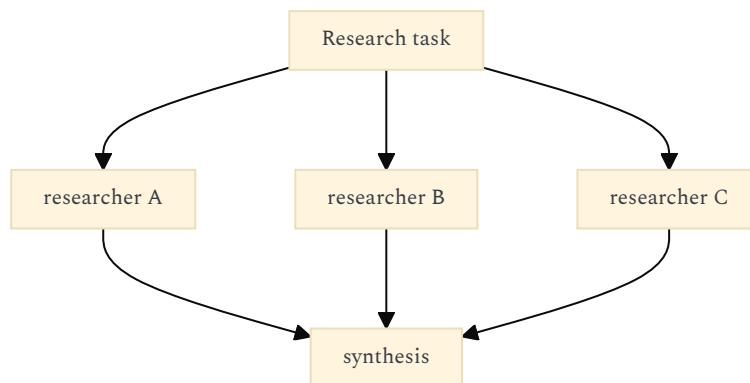
Parallelism should not require every individual primitive to understand concurrency.
A composition can determine whether several independent operations should execute:

serially

or:

in parallel

This keeps the underlying primitives simple.
It also becomes useful for agents:



The orchestration layer can decide that A, B and C are independent while synthesis depends on all three.

41. PROJET COMPLEXE ASC IS WHERE THIS BECOMES PRACTICAL

Projet Complexe ASC can define compositions such as:



These are not ASC primitives.
They are meaningful compositions for this particular environment.
This is exactly why the third project should remain thin.

42. PROJET COMPLEXE CAN THEREFORE EVOLVE WITHOUT REDEFINING ASC

Suppose Projet Complexe eventually needs:

literature-review

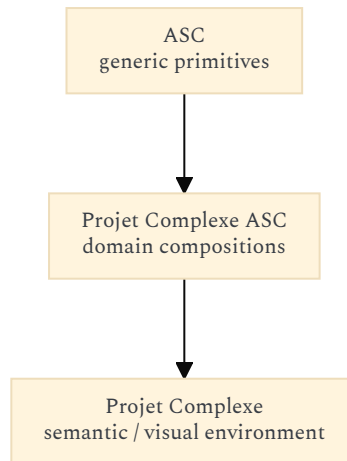
It can compose existing capabilities.
Suppose it later needs:

experiment

Again, compose capabilities.
Suppose it needs:

```
publish
```

Again, compose capabilities.
The architecture becomes:



This is much healthier than adding every new second-brain concept to ASC itself.

43. THE REPOSITORY BOUNDARIES

The resulting repository ecosystem should be approximately:

```
github.com/Paulmicha/asc
```

ASC core

Generic computational vocabulary and mechanisms.

Contains:

- entities
- declarations
- namespaces
- fields
- props
- includes
- abilities/contracts
- sidecars
- entry points
- hooks
- DSL
- Builder
- threads
- generic execution mechanisms

It should remain domain-agnostic.

```
github.com/Paulmicha/projet-complexe-asc
```

Projet Complexe ASC

Specific ASC compositions and pivots for the Projet Complexe environment.

Contains:

- project-specific entry points
- domain-specific hooks
- knowledge-related compositions
- task-related compositions
- indexing pivots
- research pivots
- agent pivots
- environment declarations
- integration glue

It should remain thin enough that its concepts can be removed without damaging ASC.

```
github.com/Paulmicha/projet-complexe
```

Projet Complexe

Desktop and semantic environment.

Contains:

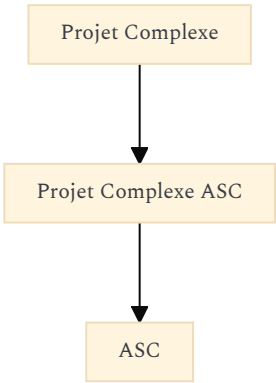
- Tauri
- SolidJS
- task model
- knowledge model
- project model
- research model
- agent visualization
- graph visualization
- document interfaces
- activity timeline
- semantic interpretation
- task/knowledge control logic

It should not contain generic OS abstractions.

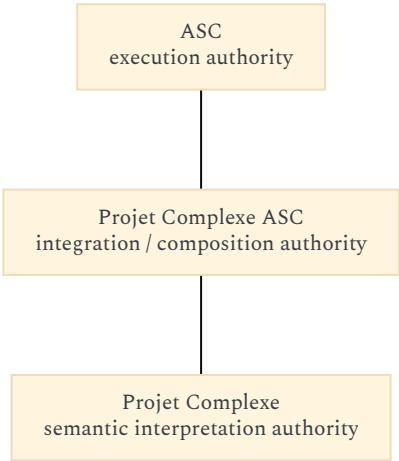
44. THE THREE LAYERS CAN BE SUMMARIZED IN ONE TABLE

Project	Fundamental question	Vocabulary	Responsibility
ASC	What exists and what can be done?	entities, pivots, hooks, capabilities, execution	generic computational substrate
Projet Complexe ASC	Which ASC capabilities does this environment expose, and how?	specific entry points, compositions, integrations	domain-specific ASC layer
Projet Complexe	What am I trying to accomplish, what do I know, and what does it mean?	tasks, knowledge, projects, research, agents	semantic + visual environment

The dependency direction is:

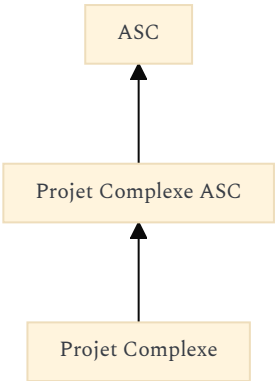


But the conceptual authority is different:

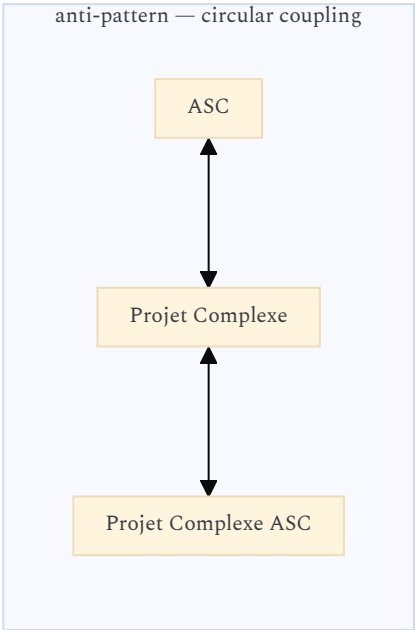


45. THE ENVIRONMENT IS DELIBERATELY ASYMMETRIC

The three projects should not know equally much about one another.
ASC should know nothing about Projet Complexe.
Projet Complexe ASC knows ASC and Projet Complexe's operational requirements.
Projet Complexe knows the semantic concepts it needs and the capabilities exposed to it.
Therefore:



not:



This one-way dependency is what keeps the architecture from becoming circular.

46. THE SECOND BRAIN SHOULD NOT BECOME AN ASC FRONTEND

This sounds like a semantic distinction, but it has practical consequences.
A generic ASC client might expose:

```
filesystem
processes
machines
services
workers
```

Projet Complexe should instead expose:

```
tasks
knowledge
projects
sources
research
agents
```

It can *show* machines, processes and services when they become relevant.
But those are not its organizing principles.
For example:

This task is blocked because the required service is unavailable.

is a Projet Complexe interpretation of ASC state.
The UI should not become:

Here is a service management console.

47. CONVERSELY, ASC SHOULD NOT BECOME A KNOWLEDGE GRAPH

ASC can expose relationships.
It can model entities.
It can make computational structures discoverable.
But it should not acquire a second-brain-specific ontology such as:

idea
belief
argument
literature review
research question
publication
epistemic confidence

unless such concepts eventually prove generic enough to belong in ASC.
The default assumption should be:

domain semantics belong above ASC.

This keeps ASC genuinely reusable.

48. THE ARCHITECTURE IS RECURSIVE

The most interesting property of the whole system is that the same pattern can repeat.
ASC can represent:

machines
projects
software
workers
agents

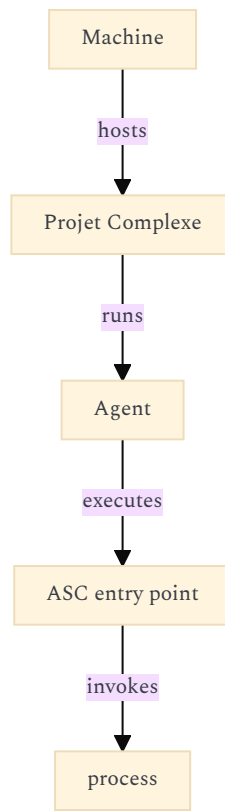
Projet Complexe can represent:

tasks
projects
knowledge
agents

Projet Complexe ASC can compose ASC capabilities into:

research
indexing
execution
agent orchestration

And the result can itself be represented by ASC.
For example:



The system can therefore describe itself.
That is potentially much more important than any individual UI feature.

49. THE SAME STRUCTURE CAN BE SEEN FROM DIFFERENT PERSPECTIVES

A computational object can be viewed as:

```
code
filesystem
entity
sidecar
DSL expression
entry point
execution
event
graph node
```

These should not necessarily be separate objects.
They can be representations of the same underlying structure.
This is one of ASC's strongest ideas.
It gives Projet Complexe something substantial to visualize.
The UI does not need to invent an artificial graph.
The graph is already implicit in the relationships between computational objects.

50. THIS IS WHY THE DESKTOP UI CAN BECOME MUCH MORE THAN A DASHBOARD

The Tauri application can become a visual projection of:

```
what exists
what is known
what is happening
what can happen
what needs to happen
why it needs to happen
what is currently blocking it
```

That is much closer to a **second brain** than a conventional productivity application.
And because ASC makes the computational environment observable, the second brain does not have to remain purely informational.
It can act.

51. BUT ACTION SHOULD REMAIN MEDIATED BY ASC

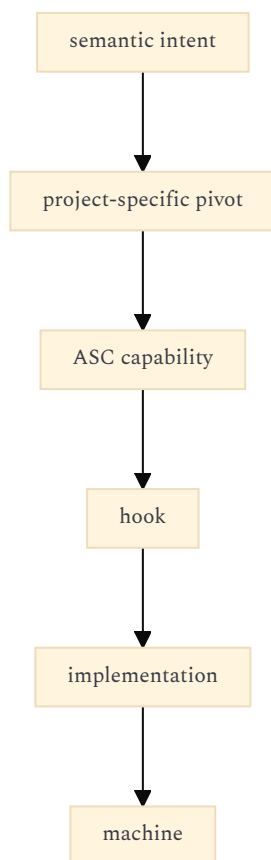
The core invariant remains:

Tauri never directly operates the host. It asks ASC to operate the host.

More broadly:

Projet Complexe never needs to know how a computational action is implemented. It asks for a capability or pivot.

This provides:



The same operation can therefore remain reproducible from the terminal.

52. THIS IS ALSO IMPORTANT FOR AGENTS

An autonomous agent should not receive an arbitrary shell and an enormous unstructured environment and be expected to infer everything. It should be able to discover:

```
entities
capabilities
entry points
constraints
relationships
available knowledge
unknown knowledge
```

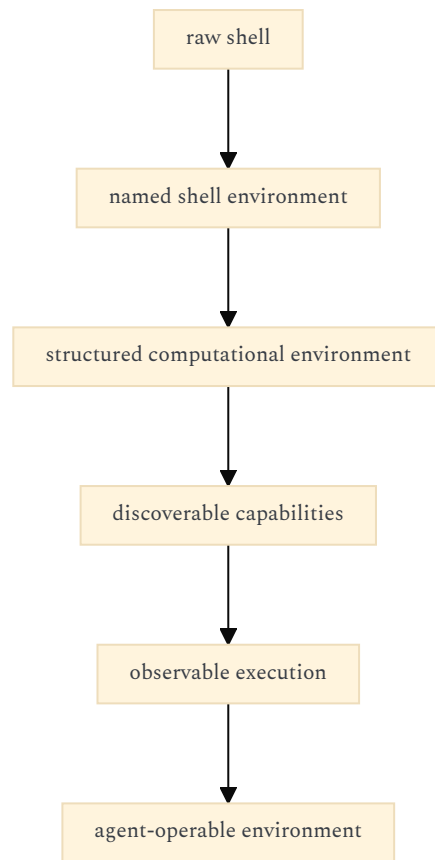
Then it can reason over a structured environment.

The goal is not to prevent agents from using the shell.

The goal is to make the shell's affordances legible.

53. ASC AS AN ENVIRONMENT FOR AGENTS

This gives a useful progression:



The agent is not merely executing commands.
It is navigating a vocabulary.
This is where the original ambition of ASC becomes relevant to autonomous agents.

54. THE GO ANALOGY

The Go analogy should remain.
ASC is not literally a game, and the analogy is not about gamification.
It is about the relationship between a small vocabulary and a huge combinatorial space.
Go has relatively few fundamental concepts:

stones
intersections
groups
liberties
territory
connections
captures

Yet enormous structures emerge from their relationships.
ASC is attempting something analogous for the computational environment:

entities
entry points
namespaces
environments
scripts
sidecars
hooks
variants
arguments
composition

The important property is not that there are few concepts.
It is that the concepts have stable meanings and can be recombined.
This gives rise to a large space of possible computational structures without requiring a new abstraction for every individual project.
In that sense:

ASC is trying to make the computational environment playable.

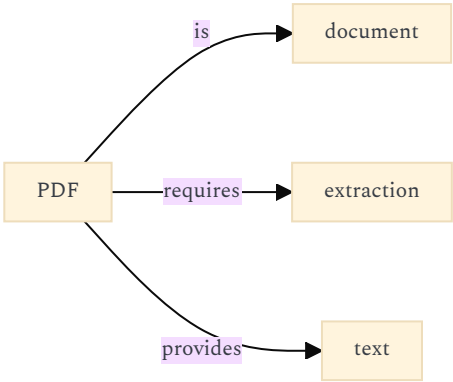
Not because computation is a game.
Because a stable vocabulary turns an otherwise chaotic space of possibilities into a space of meaningful moves.

55. “IF YOU NAME THINGS RIGHT, PROJECTS PRACTICALLY WRITE THEMSELVES”

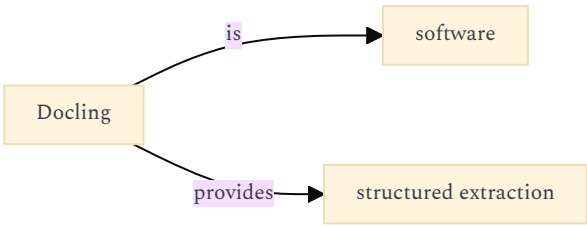
This principle becomes much clearer in the three-project architecture.
Naming does not mean inventing arbitrary labels.
It means establishing stable relationships between:

```
what something is
where it exists
what it can do
how it can be invoked
how it can be composed
what it produces
what depends on it
```

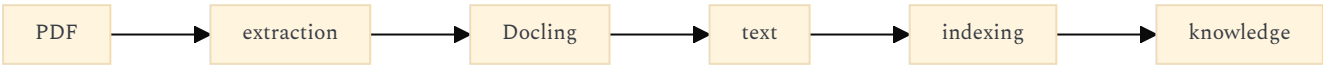
Once those relationships are explicit, a surprising amount of implementation becomes composition.
For example:



and:



The system can then compose:



The project is not written from scratch.
It is assembled from named capabilities.

56. THIS IS THE REAL RELATIONSHIP BETWEEN ASC AND THE SECOND BRAIN

The Second Brain does not need ASC because it needs a backend.
It needs ASC because it needs a **computational vocabulary**.
Conversely, ASC does not need the Second Brain in order to be useful.
The Second Brain is simply one particularly interesting environment in which the consequences of ASC's vocabulary become visible.
That distinction matters.
It means ASC can continue evolving independently.
And Projet Complexe can become increasingly sophisticated without forcing ASC to absorb its semantics.

57. THE RESEARCH ENVIRONMENT BECOMES A PARTICULARLY INTERESTING TEST CASE

Projet Complexe can contain:

sources
documents
concepts
questions
claims
tasks
agents
experiments
publications

ASC can provide:

files
processes
workers
indexers
extractors
LLMs
containers
remote hosts

Projet Complexe ASC composes them:

research
index
extract
search
relate
summarize
compare
publish

The desktop application then exposes the resulting system visually.
This gives the project a coherent path from:

document

to:

knowledge

to:

task

to:

agent

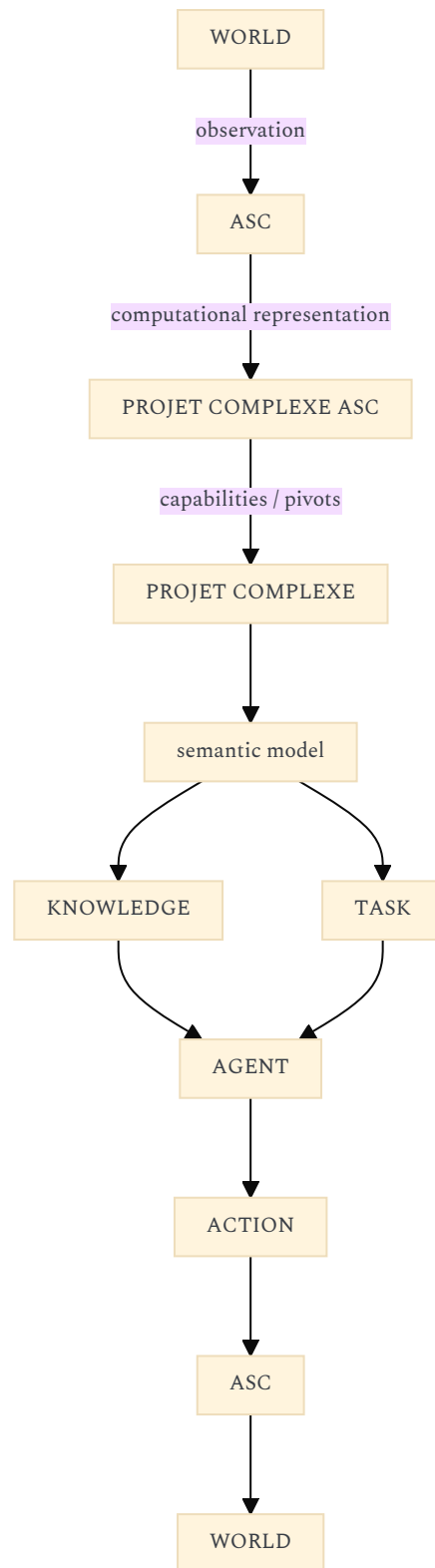
to:

execution

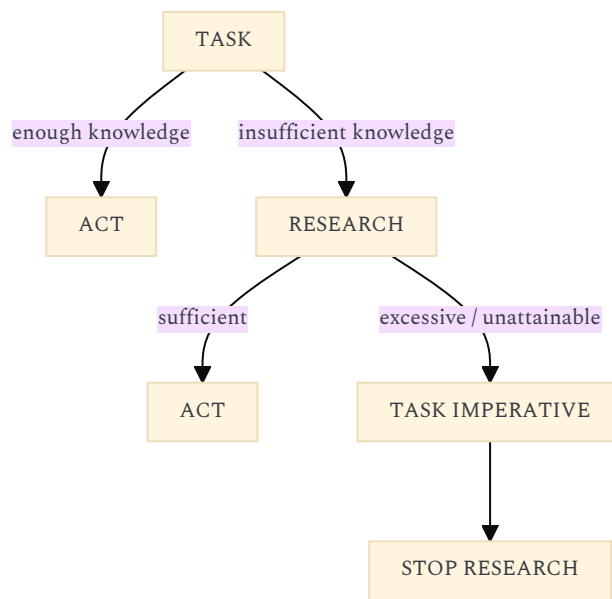
to:

new knowledge

58. THE WHOLE ARCHITECTURE CAN BE EXPRESSED AS A LOOP



And the task/knowledge mutual killswitch operates inside this loop:



This is a much more precise model of autonomous work than simply:

agent → tools → result

59. WHAT SHOULD BE BUILT FIRST

The first milestone should not be:

Build the complete Second Brain.

Nor:

Build the complete agent framework.

Nor:

Build the complete graph database.

The first milestone should establish the architectural invariant:

A useful operation can be represented as a stable ASC pivot, executed from the terminal, and consumed by Projet Complexe without the UI knowing its implementation.

A minimal example might be:

```

ASC
  expose machine/project state

Projet Complexe ASC
  expose a few useful pivots

Projet Complexe
  display and invoke them
  
```

For example:

```

Machine
  Debian
  CPU
  memory
  GPU

ASC
  running

Projects
  projet-complexe
  projet-complexe-asc
  asc

Workers
  indexing
  idle
  
```

If the same operation is reproducible through:

```
terminal → ASC
```

and:

```
Tauri → Projet Complexe ASC → ASC
```

the most important architectural boundary has already been demonstrated.
Everything else can grow around it.

60. WHAT SHOULD NOT BE BUILT FIRST

Avoid premature construction of:

```
a giant API
a giant graph schema
a giant agent framework
a giant frontend component library
a second execution engine
a second shell abstraction
a second task manager
a second knowledge database
```

The architecture becomes stronger by keeping the primitives small.

The central question should repeatedly be:

Is this a new primitive, or is it a composition of existing primitives?

If it is a composition, it probably belongs above ASC.

If it is a genuinely reusable computational primitive, it may belong in ASC.

If it is specific to Projet Complexe, it probably belongs in Projet Complexe ASC.

If it exists primarily to interpret, visualize or organize meaning for the user, it belongs in Projet Complexe.

61. A PRACTICAL RULE FOR DECIDING WHERE SOMETHING BELONGS

When introducing a new concept:

Put it in ASC if:

- it describes a generic computational thing;
- it can be useful independently of Projet Complexe;
- it concerns naming, addressing, composition or execution;
- it is sufficiently domain-independent;
- it can support multiple consumers.

Put it in Projet Complexe ASC if:

- it composes generic ASC capabilities;
- it is specific to the Projet Complexe environment;
- it defines a useful entry point or pivot for that environment;
- it connects concrete tools to semantic capabilities;
- removing it would not damage ASC itself.

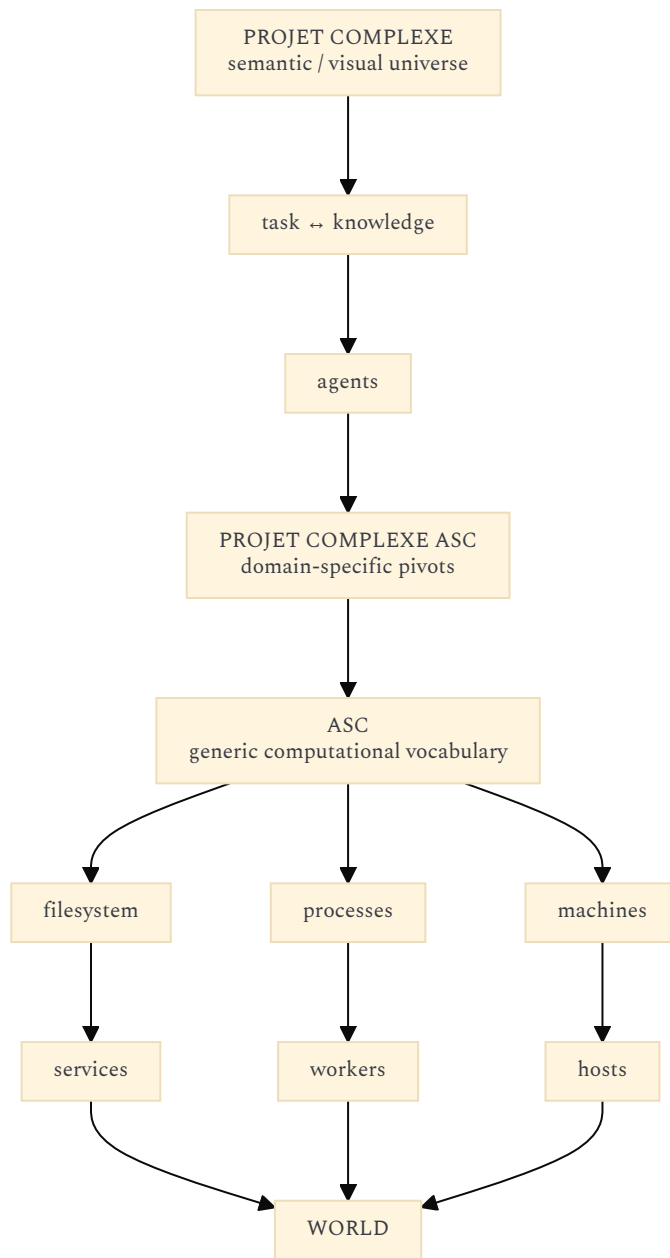
Put it in Projet Complexe if:

- it represents meaning, knowledge or intention;
- it belongs to the task/knowledge model;
- it concerns research, concepts, sources or publications;
- it interprets ASC events;
- it is primarily a human/agent semantic interaction.

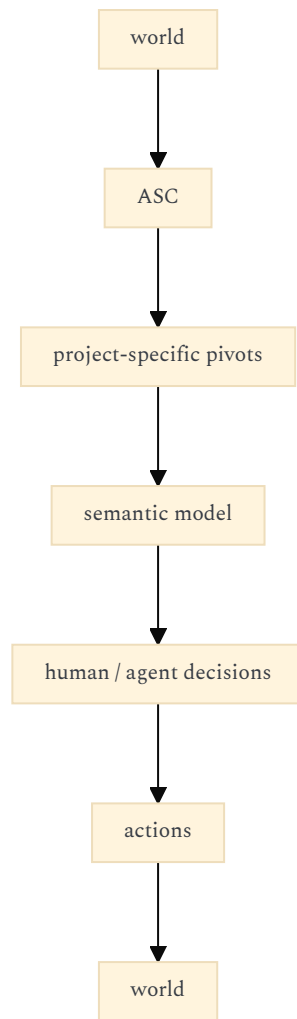
This rule is more useful than deciding according to programming language or repository convenience.

62. THE ARCHITECTURAL HIERARCHY

The resulting system can be summarized as:



The direction of abstraction is:



63. THE DEEPER AMBITION

ASC begins from the hard problem of naming things.

But the problem becomes more interesting once names are not isolated labels.

A useful name can become:

```
address
pivot
capability
relationship
composition point
semantic reference
execution target
```

That means naming can become a bridge between:

```
filesystem
shell
software
machines
projects
knowledge
agents
```

The same vocabulary can progressively connect things that are normally represented through completely different systems.

This is the real potential of the architecture.

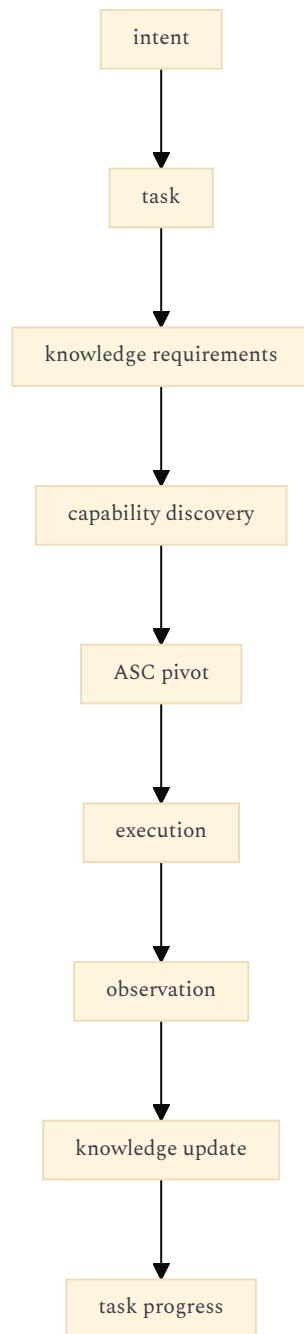
64. FROM SHELL VOCABULARY TO AGENT VOCABULARY

A shell gives us operations.

ASC attempts to give those operations stable names and structures.

Projet Complexe gives those structures semantic context.

Agents can then operate somewhere between the two:



This is much closer to an environment in which agents can genuinely operate than simply giving an LLM access to a terminal. The terminal remains there. It is simply no longer the only representation of what the terminal can do.

65. THE ENVIRONMENT BECOMES INSPECTABLE

One of the most important consequences is that the system becomes inspectable at multiple levels. A human can see:

```
project
task
knowledge
agent
machine
```

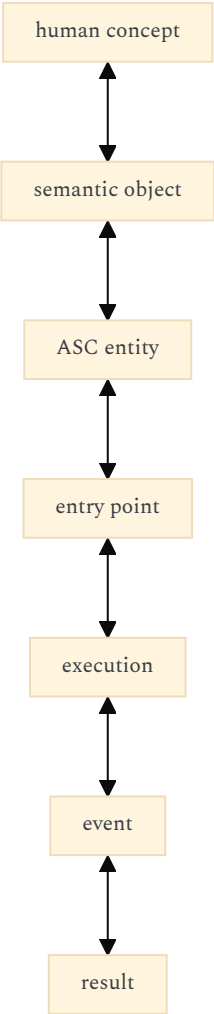
An agent can discover:

```
entity
capability
entry point
constraint
event
```

The shell can execute:

```
command
script
process
worker
```

And ASC can connect those representations.
This means the same activity can be inspected as:



That traceability is potentially more important than any individual feature.

66. THE SYSTEM SHOULD REMAIN LIGHTWEIGHT

The architecture does not require a huge stack.
The desktop application can remain technologically simple:

```
Tauri 2
SolidJS
TypeScript
Vite
Kobalte
plain CSS
```

The backend does not need to be a monolithic server.
ASC can orchestrate:

```
Bash
Rust
Python
external binaries
Docker
remote commands
```

depending on what is appropriate.
The semantic layer can use whatever storage is appropriate:

```
files
SQL
search index
graph database
event log
```

The point is not technological maximalism.

The point is to establish a coherent vocabulary over heterogeneous technologies.

67. THE RESULTING SYSTEM IS DELIBERATELY PLASTIC

This architecture allows substitutions without conceptual collapse.

For example:

```
Solr → another search engine
Docling → another extractor
ArangoDB → another graph backend
local LLM → remote model
agent runtime A → agent runtime B
Debian → another OS
Tauri → another client
```

The important concepts remain:

```
Search
Extract
Graph
Agent
Machine
Task
Knowledge
Project
Entry point
Capability
```

This is the kind of plasticity that makes the architecture durable.

68. THE KEY INVARIANT

The entire system can ultimately be reduced to one principle:

Do not couple semantic meaning to implementation details when a stable computational pivot can separate them.

For example:

```
"extract this document"
```

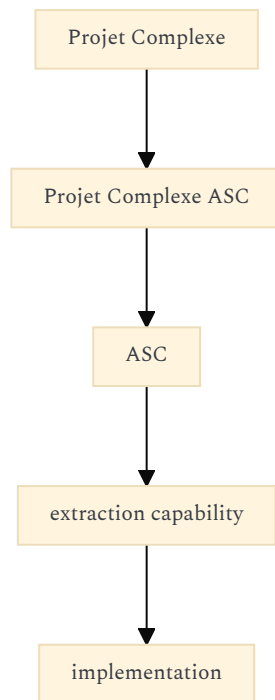
should not mean:

```
"run Docling in this Docker container using this exact command"
```

It should mean:

```
extract-document
```

which can resolve through:



The same applies to:

```
search
index
publish
run
stop
research
spawn
inspect
```

Stable names make implementations replaceable.

69. THE DEEPER MEANING OF “LET'S MAKE WORDS MATTER”

This phrase is not merely a slogan.

It describes the architecture.

If a word such as:

```
agent
```

has no stable meaning, it is merely a label.

If `agent` has:

```
a declaration
a namespace
a representation
a capability set
an execution pivot
an event stream
a lifecycle
```

then the word becomes operational.

Likewise:

```
task
knowledge
machine
project
document
service
worker
```

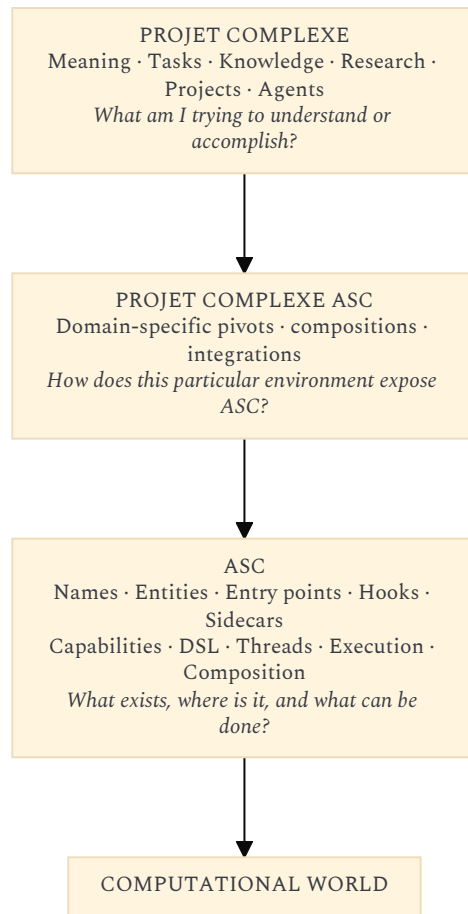
can become computable concepts rather than merely UI labels.

The goal is therefore:

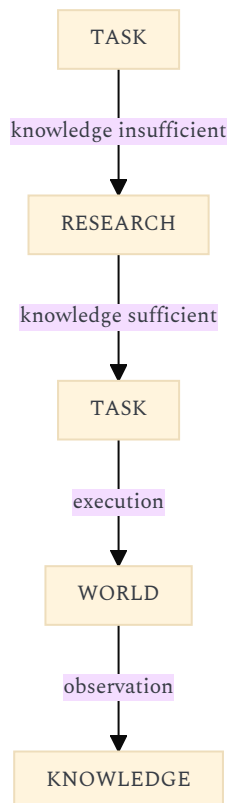
Make words matter by giving names stable computational consequences.

70. THE FINAL SYNTHESIS

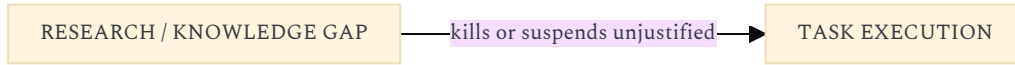
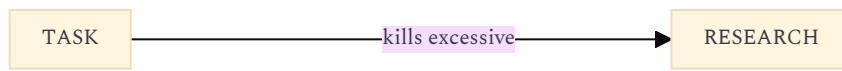
ASC, Projet Complexe ASC and Projet Complexe should not be understood as three versions of one application. They form a stack of increasingly specific meaning.



The relationship between task and knowledge then becomes the dynamic core of the upper layer:



with a mutual killswitch:



This gives autonomous agents something more interesting than unrestricted tool access.
It gives them a bounded ecology of:

intent
knowledge
uncertainty
action
observation
change
execution

ASC provides the computational vocabulary in which that ecology can operate.
Projet Complexe ASC provides the concrete pivots through which this particular environment is assembled.
Projet Complexe provides the semantic environment in which humans and agents can understand, organize and direct it.

GENERICITY SCALE AND THE THREE-PROJECT BOUNDARY

The three-project architecture should not be understood as three successive abstraction layers.

There are actually **two dimensions**:

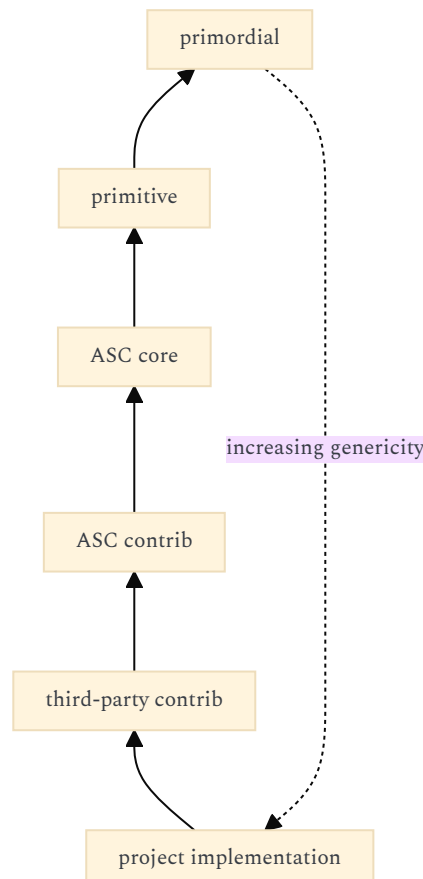
1. a **genericity scale**, describing how broadly reusable a concept is;
2. a **project boundary**, describing which project currently owns and composes that concept.

The genericity scale is:

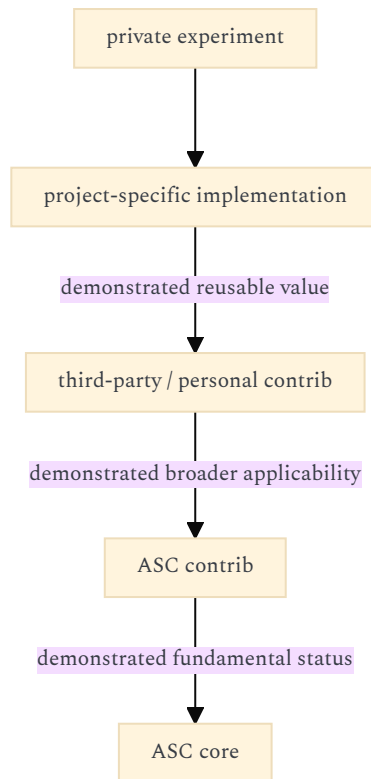
1. primordial
2. primitive
3. ASC core extension
4. ASC contrib extension
5. third-party contrib extension
6. project-specific implementation

This distinction matters because a project-specific concept can still be implemented *using* ASC. Conversely, something originally invented for Projet Complexe may later prove sufficiently reusable to migrate upward.

The intended evolutionary direction is therefore:



Or operationally:



This means that **ASC should not try to predict the final ontology in advance.**

A concept can start as a very specific implementation and migrate upward when experience demonstrates that it deserves to become generic.

THE SIX LEVELS

1. PRIMORDIAL

These are concepts so fundamental that ASC builds its vocabulary around them rather than defining them as ordinary extensions. They form the substrate from which the rest of the system is constructed.

2. PRIMITIVE

Primitives are the basic reusable computational concepts of ASC. They should have stable semantics and should not depend on Project Complex. Examples include the kinds of concepts around:

```
entity
field
prop
able
include
namespace
entry point
hook
sidecar
thread
change
```

The exact primitive vocabulary can evolve, but its defining property is **genericity**.

A primitive should make sense independently of a particular second brain, research workflow or desktop application.

3. ASC CORE EXTENSIONS

These are concepts that are not necessarily primordial or primitive, but have become sufficiently fundamental to belong to ASC itself. They extend the generic computational vocabulary without being tied to one particular project. ASC core is therefore not merely a fixed kernel.

It can grow.

The important constraint is that its additions remain broadly meaningful.

4. ASC CONTRIB EXTENSIONS

These are reusable extensions that are valuable within the ASC ecosystem but are not fundamental enough to belong to the core. This provides an important pressure-release mechanism. Without a contrib level, every useful abstraction faces a binary decision:

```
core
or
not ASC
```

The contrib level allows:

```
useful + reusable
but
not fundamental
```

to remain part of the ASC ecosystem without contaminating the core vocabulary.

5. THIRD-PARTY CONTRIB EXTENSIONS

This is where reusable external implementations can live.
A capability may originate outside ASC itself while still exposing itself through ASC's vocabulary.
For example, a particular document-extraction, indexing, graph, AI or operating-system integration can become an ASC-compatible extension without becoming an ASC primitive.
This is where tool-level portability becomes particularly useful.
The semantic capability can remain stable:

```
document extraction
```

while implementations vary:

```
Docling
Tika
pdftotext
custom extractor
```

The implementation is replaceable because the capability has a stable ASC-facing address.

6. PROJECT-SPECIFIC IMPLEMENTATIONS

At the most specific end are implementations whose meaning depends on one particular environment.
This is where **Projet Complexe** ASC primarily lives.
For example:

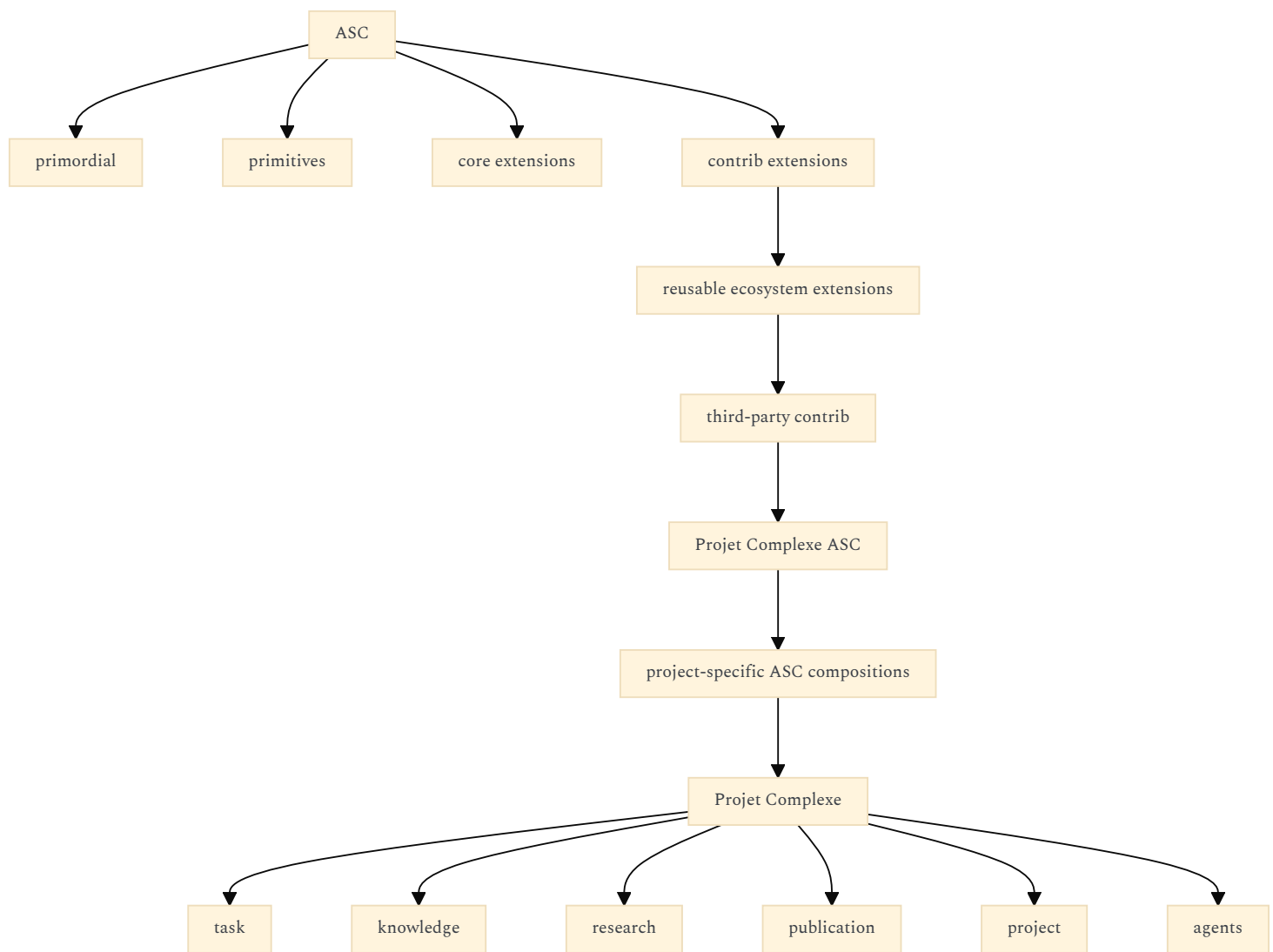
```
research
researcher-agent
knowledge-gap
task-blocked
task-resume
index-my-corpus
publish-research-note
```

may be meaningful and useful within Projet Complexe without being appropriate ASC primitives.
They can nevertheless be expressed *through* ASC.
That distinction is essential:

Being implemented with ASC does not make something part of ASC's generic vocabulary.

THE THREE PROJECTS MAPPED ONTO THE GENERICITY SCALE

The three projects therefore occupy different roles.



But the last two should not be interpreted as simply “levels 6 and 7”.

They are different dimensions.

Projet Complexe ASC is primarily a *computational integration/composition space*.

Projet Complexe is primarily a *semantic and visual application space*.

A project-specific concept can therefore exist in both:

Projet Complexe ASC:
 spawn-researcher
 index-document
 search-knowledge
 execute-task

Projet Complexe:
 Research
 KnowledgeGap
 Task
 Agent
 Evidence
 Decision

The first group describes **how the environment operates**.

The second describes **what that operation means**.

WHY THE BOUNDARY MATTERS

The genericity scale prevents a common architectural failure:

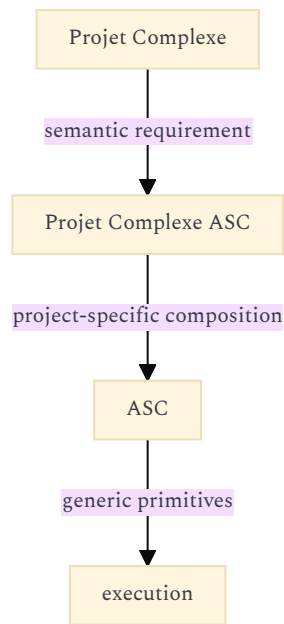
promoting every useful project-specific abstraction into the generic core.

Suppose Projet Complexe needs:

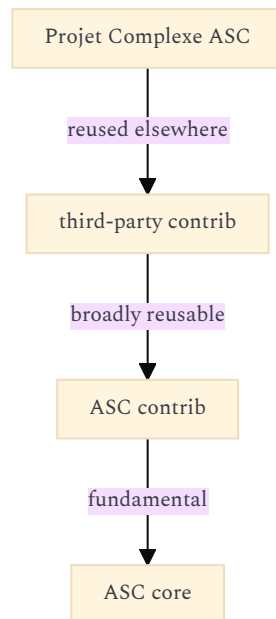
research-agent

That does not mean ASC should acquire a primitive called `research-agent`.

Instead:



Later, if the abstraction proves useful outside Projet Complexe, it can migrate:



This makes the architecture evolutionary rather than speculative.

GENERICITY IS THEREFORE A PROMOTION MECHANISM

The six levels should not merely classify existing things.

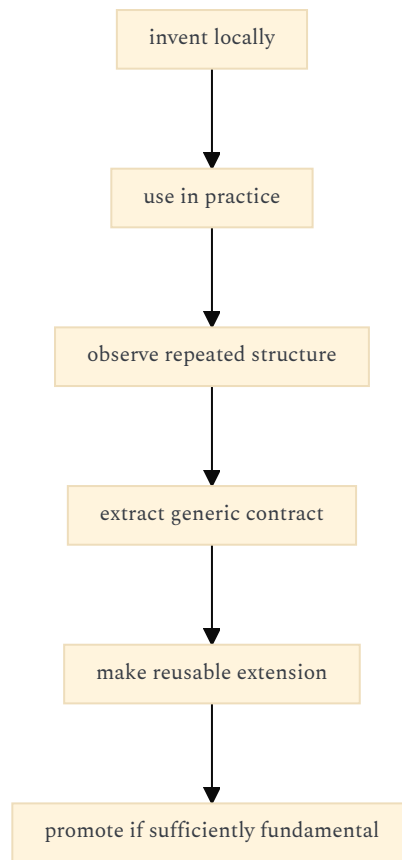
They should describe how concepts **mature**.

A project can safely invent concepts locally.

It does not need to know whether they belong in ASC.

Experience determines that.

A useful pattern is:



This is especially important for autonomous-agent concepts.
It is entirely reasonable for Projet Complexe to experiment with:

knowledge-gap
research-budget
research-killswitch
task-killswitch
agent-run
evidence-sufficiency

without deciding prematurely that these are universal concepts.
The project is precisely where such concepts can be tested.
If they later prove to describe a more general class of autonomous systems, they can migrate upward.

THE THREE PROJECTS THEREFORE HAVE DIFFERENT AMBITIONS

ASC CORE

Ambition:

establish a generic computational vocabulary.

ASC should answer:

What is this?
Where is it?
What does it expose?
What can it do?
How can it be composed?
How can it be executed?
How can its effects be observed?

Its success criterion is **genericity**.

PROJET COMPLEXE ASC

Ambition:

turn generic ASC mechanisms into the specific computational pivots required by one sophisticated environment.

It should answer:

How do we compose ASC to provide:
research?
indexing?
agent execution?
knowledge operations?
task operations?

Its success criterion is **useful composition without contaminating ASC's generic core**.

PROJET COMPLEXE

Ambition:

provide the semantic and visual environment in which humans and agents organize tasks, knowledge and action.

It should answer:

What are we trying to accomplish?
What do we know?
What do we not know?
What matters?
What is related?
What should happen next?
Why is execution blocked?
When is knowledge sufficient?

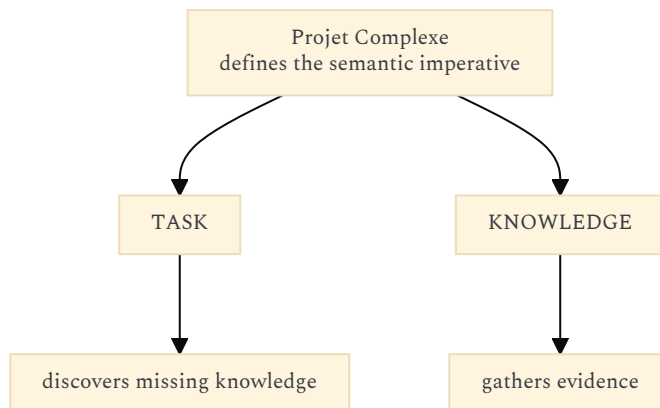
Its success criterion is **semantic usefulness**.

THE MUTUAL KILLSWITCH BELONGS PRIMARILY HERE

The task/knowledge mutual killswitch is therefore not an ASC primitive by default.

It is initially a **Projet Complexe autonomous-agent control concept**, potentially implemented through Projet Complexe ASC and ASC primitives.

The relationship is:



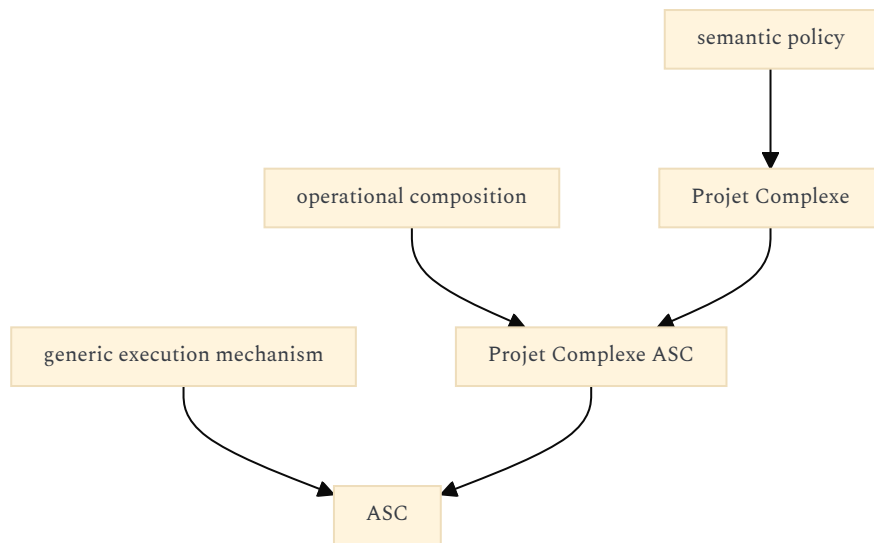
Projet Complexe ASC can expose the operational pivots:

spawn-researcher
pause-task
resume-task
kill-research
inspect-knowledge-gap

ASC supplies the generic mechanisms:

thread
change
command
hook
entry point
event
stop
execute

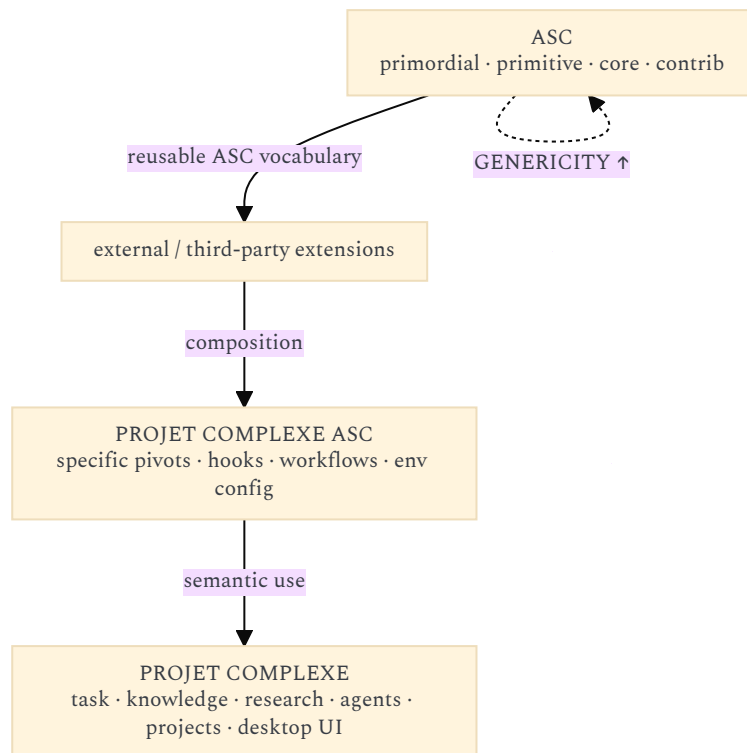
Thus:



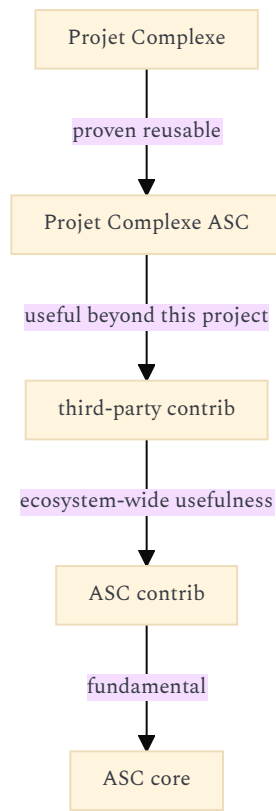
This is exactly the sort of concept that the genericity scale is designed to protect.
 The **idea** may eventually prove highly general.
 The **implementation** should first remain project-specific.

THE FINAL ARCHITECTURAL PICTURE

The most accurate representation is therefore not a simple vertical stack.
 It is:



And concepts can move **upward in genericity** when experience warrants it:



That is the missing conceptual mechanism tying the three projects together.

ASC is not merely the bottom layer. It is the genericity gradient's stable center. Projet Complexe ASC is the experimental/compositional frontier.

Projet Complexe is the semantic application where new abstractions are discovered and tested.

Confidence level: 0.99

CONCLUSION

ASC starts from a deceptively simple problem:

The hard problem of naming things.

Its ambition is to establish a common, shared vocabulary for anything interacting with the shell somehow: files, directories, processes, machines, services, scripts, actions, environments, dependencies, workflows and eventually higher-level computational structures.

The ambition is not to replace the shell.

It is to make its vocabulary explicit.

An entity gives something a name.

An entry point gives it a stable pivot.

A namespace gives that name context.

A sidecar gives a virtual entity a concrete representation.

A capability describes what something can participate in.

A hook provides a concrete implementation.

A variant provides an alternative implementation.

A wrapper or nester composes existing behavior.

YAML declares.

Paths address.

The DSL formulates.

Entry points pivot.

Hooks implement.

Sidecars concretize.

The Builder generates.

The shell executes.

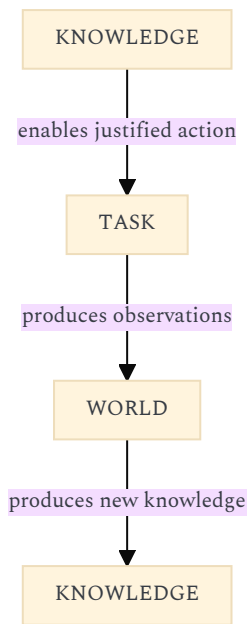
And Projet Complexe eventually gives the whole structure another representation:

```
graph
timeline
task
knowledge
agent
project
machine
```

The same structure can therefore be viewed as code, filesystem, entity, sidecar, DSL, generated artifact, execution, event or graphical object depending on the perspective.

That is where the three-project architecture becomes coherent.

ASC remains generic.
Projet Complexe ASC provides the specific pivots needed to turn generic ASC capabilities into a concrete environment.
Projet Complexe becomes the semantic and visual environment that gives those capabilities meaning.
The Second Brain does not replace ASC.
It interprets it.
ASC does not become the Second Brain.
It makes the computational world legible to it.
And Projet Complexe ASC is the narrow bridge between the two.
This also explains why the task/knowledge distinction matters so much.
A purely task-oriented agent risks acting without understanding.
A purely knowledge-oriented agent risks researching without end.
The interesting autonomous system lies between them:



The mutual killswitch keeps the loop bounded.
A task can stop itself when it discovers that knowledge is missing.
Knowledge gathering can stop itself when the task imperative makes further research unjustified, excessive or unattainable.
The objective is neither:

act at all costs

nor:

know everything before acting.

It is:

know enough to make the next move, and keep moving toward something worth accomplishing.

This is where the Go analogy becomes useful.
Go has a relatively small vocabulary:

stones
intersections
groups
liberties
territory
connections
captures

Yet those primitives generate an enormous space of possible configurations.
ASC attempts something analogous for the computational environment:

entities
entry points
namespaces
environments
scripts
sidecars
hooks
variants
arguments
composition

The important property is not merely having a small vocabulary.

It is having a vocabulary whose elements have stable meanings and can be recombined.
In that sense:

ASC is trying to make the computational environment playable.

Not because computation is literally a game, but because a stable vocabulary turns an otherwise unbounded space of possibilities into a space of meaningful moves.

And this becomes particularly interesting when the player is no longer necessarily human.

That may ultimately be the bridge between ASC and autonomous agents.

An agent does not merely need a terminal.

It needs an environment in which things have names, capabilities, relationships, constraints, histories and possible actions.

It needs to know not only:

“What command can I execute?”

but:

“What exists?”

“What can I do?”

“What do I need to know?”

“What is currently preventing me from acting?”

“Have I learned enough?”

“What changed because I acted?”

ASC can provide the computational substrate for those questions.

Projet Complexe can provide the semantic substrate.

Projet Complexe ASC can provide the pivots that connect them.

The resulting ambition is therefore larger than a desktop application, a shell framework or an agent controller considered separately.

It is an attempt to make the computational environment sufficiently:

nameable
addressable
composable
observable
interpretable

that increasingly complex projects can emerge from a relatively small set of shared primitives.

If you name things right, projects practically write themselves.

Let's make words matter.